



Optimal resource management for multi-access edge computing without using cross-layer communication

Ankita Koley^{*}, Chandramani Singh

Department of Electronic Systems Engineering, Indian Institute of Science, Bangalore, 560012, Karnataka, India

ARTICLE INFO

Keywords:

Multiaccess edge computing
Cloud computing
Scheduling
Throughput optimality
Drift-plus-penalty

ABSTRACT

We consider a Multi-access Edge Computing (MEC) system with a set of users, a base station (BS) with an attached MEC server, and a cloud server. The users can serve the service requests locally or can offload them to the BS which in turn can serve a subset of the offloaded requests at the MEC and can forward the requests to the cloud server. The user devices and the MEC server can be dynamically configured to serve different classes of services. The service requests offloaded to the BS incur offloading costs and those forwarded to the cloud incur additional costs; the costs could represent service charges or delays. Aggregate cost minimization subject to stability warrants optimal service scheduling and offloading at the users and the MEC server, at their application layers, and optimal uplink packet scheduling at the users' MAC layers. Classical back-pressure (BP) based solutions entail cross-layer message exchange, and hence are not viable. We propose virtual queue-based drift-plus-penalty algorithms that are throughput optimal, and achieve the optimal delay arbitrarily closely but do not require cross-layer communication. We first consider an MEC system without local computation, and subsequently, extend our framework to incorporate local computation also. We demonstrate that the proposed algorithms offer almost the same performance as BP based algorithms. These algorithms contain tuneable parameters that offer a trade off between queue lengths at the users and the BS and the offloading costs.

1. Introduction

Digital applications, e.g., navigation and localization, video streaming, face recognition, augmented reality, gaming, etc., entail executing certain computation tasks. These tasks are run on users' devices, e.g., mobiles, or using *cloud computing*. Users' machines have limited power and compute and storage resources. Hence, executing the computation tasks at these machines consumes disproportionate power and incurs significant latency which impact the users' experience. Cloud computing serves as a cost and energy efficient alternative to local computation. Most of the digital applications have been running using the cloud as the primary compute and storage resource over the last decade [1]. However, compute and storage demands are likely to increase manyfold in near future, e.g., Cisco has predicted that there will be 500 billion networked devices by 2030 [2]. Moreover, data centers will consume almost 15 times more energy in 2030 than their current consumption [3]. Hence, solutions solely based on cloud computing would consume extensive network resources, and will also incur huge delays, negating the benefits of cloud computing [4]. Multi-access Edge Computing (MEC) is an emerging technology in which cloud computing services are extended to edge of the Internet, i.e., wireless access points (APs) and base stations (BSs). Edge computing facilitates energy efficient and quicker execution of resource intensive tasks of users without incurring significant network resource consumption and large latencies [5].

^{*} Corresponding author.

E-mail address: ankitakoley@iisc.ac.in (A. Koley).

MEC, specifically when it involves wireless edge networks, brings along new challenges, a few of which are as follows.

- (a) The edge devices, unlike the centralized cloud, have limited compute and storage capability.
- (b) Many contemporary services require sizable data to be pre-stored at the edge devices.
- (c) At any instant, different services have different workloads (in bits). Moreover, they also need different amounts of CPU processing resources per unit of workload.
- (d) Depending on the instantaneous network loads, wireless channel conditions, and resource allocation protocols, the users may receive different Quality of Service (QoS), e.g., throughput, latency etc.

Given the dynamism of various attributes, it is imperative that one judiciously decides which tasks to offload and where should these be offloaded, i.e., to the MEC server or to the cloud. In particular, one has to deal with the stochasticity of user requests, wireless environment and communication constraints, compute resource allocation at the MEC server, and the delay incurred in processing the services at the cloud. Observe that the users' devices make offloading decisions at application layer and communication scheduling decisions at MAC layer. Subsequently, the BS also makes offloading decisions (regarding offloading to the cloud) at application layer. The MEC and cloud systems can be modeled as multihop networks and offloading and scheduling problems can be posed as joint routing and scheduling problems. Furthermore, their distributed solutions can be derived using classical back-pressure and drift-plus-penalty frameworks [6]. However, these solutions would require message exchange across successive hops, i.e., between application and MAC layers in our context. Such exchanges go against the layering principles of communication networks.

In this work, we develop offloading and scheduling algorithms that optimize the costs associated with offloading the tasks to the MEC server and to the cloud while stabilizing all the application and MAC layer queues in the system. In particular, we design algorithms that do not require cross-layer message exchange, stabilize the MEC system and minimize the delay costs.

1.1. Related work

Xu et al. [7] study joint caching and task offloading at the MEC server for energy and computation delay minimization but overlook communication scheduling between the users and the BSs. Poularakis et al. [8] optimize the number of services hosted by the BS while accounting for storage, computation, and communication constraints. However, they only address the static problem, representing a snapshot of the dynamic system. They primarily focus on complexity of the static problem and propose suboptimal solutions. Simon et al. [9] study decentralized task offloading by formulating a matching game where each user tries to minimize its energy, and the mobile network operator tries to maximize its profit. Although they consider slotted systems, decisions in each slot are independent of those in the past or the future slots, which makes the problem equivalent to a static problem. The underlying assumption is that the intervals between successive arrivals are much longer than task computation times and the offloading times to the MEC server.

Dynamic user requests and network conditions warrant dynamic task offloading algorithms. Throughput and delay are important performance metrics for task offloading algorithms. Throughput optimality or stability of queues guarantees finite average delay thanks to the Little's law. Mao et al. [10] address the problem of minimizing users' compute and communication energy subject to stability of the user queues in an MEC system consisting of multiple users and a single MEC server. Bi et al. [11] design a task offloading algorithm to maximize the aggregate computation rate of all the users under the stability of users' queues and power constraints. However, these models in [10,11] exclude the primary characteristic of MEC servers, namely, limited computation and storage resources and heterogeneous service requirements.

Cai et al. [12] consider a similar problem in a more general setup consisting of multiple users and BSs and the cloud. While their model considers per service queues at the MEC server and users with different CPU requirements and MEC servers with limited compute capacity, it ignores the different storage requirements of the services and limited storage capacity at the MEC server. Next, we discuss the issues with the solution approach of [12]. It assumes availability of the following information while making dynamic task offloading decisions at the users:

- (a) the queue lengths at the users' CPUs and at the MEC servers,
- (b) the time-varying wireless channel transmission rates.

However, observe that the queue length and the wireless channel state information reside at the application and the MAC/Physical layers, respectively. The communication networking protocols use a layered structure which generally does not allow cross-layer communication. Although present-day mobile devices have Application Programming Interfaces (APIs) that allow the applications to access wireless channel states and use these in offloading decisions, the applications still cannot control the scheduling decisions that are made at the MAC layer. On the other hand, the MAC layer does not have access to the queue length information of the application layer. So, the offloading decisions must only depend on the queue lengths at the users' CPUs and at the MEC servers, and scheduling decisions must only be governed by the time-varying wireless channel transmission rates. We do not assume that MAC layer has all the information about queue lengths at the application layer of the users and the MEC server. Moreover, the back-pressure based algorithms need to continuously exchange queue length information between the MEC servers and the users, which incurs additional resource consumption and delays. Lastly, considering the MEC server's storage constraints and computation constraints is essential in view of different classes of services requiring different resources. None of the above works account for these constraints. On the other hand, we consider the storage constraints at the MEC server and also heterogeneous services.

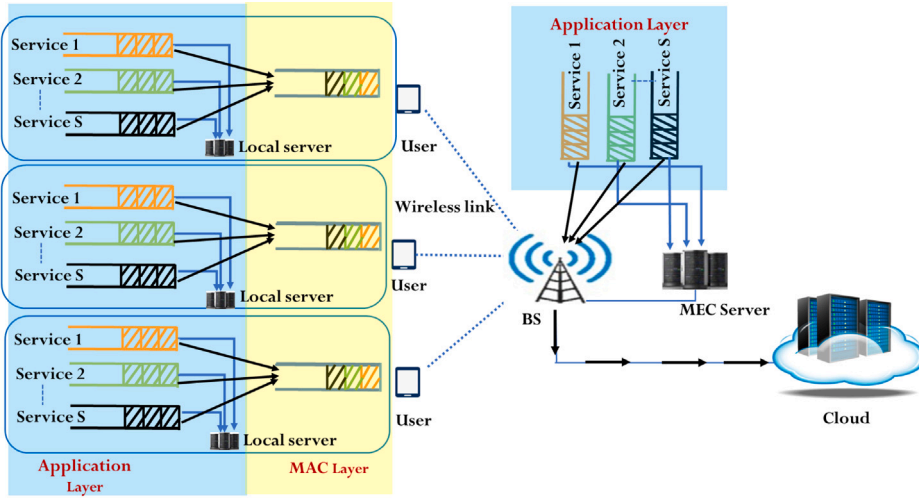


Fig. 1. Joint scheduling and task offloading in an MEC system consisting of multiple users, a BS with an MEC server, and the cloud.

Fan et al. [13] study a full offloading model considering edge–cloud collaboration where they take MEC servers’ storage and computation constraints into account and aim to minimize the average processing and offloading delays subject to stability of queues at the MEC servers and the cloud. Finally, they propose a backpressure-based algorithm that requires the queue length of all the MEC servers and the cloud. We also aim to minimize the offloading delay to the cloud subject to stability of the queues in the first part of our work. The key difference is that Fan et al. do not consider the wireless communication constraint between the users and the MEC servers. Hence, they also do not consider scheduling of packets at the users’ MAC layer, which is a crucial part of an MEC system. In the second part of our work, we aim to minimize the average cost that captures the queuing delay at the users and the MEC server, processing delay at the MEC server, and offloading delay to the cloud subject to stability of the queues. In this part, along with the wireless communication constraint, we also consider local computation, unlike [13].

Throughput-optimality has been an active research topic in the context of communication networks over the past three decades. In particular, the back-pressure algorithm which requires per flow queues and queue length information exchange across the network nodes has been proposed to stabilize multi-hop networks [14]. Max-weight, self-regulated max-weight [15], and virtual queue length based max-weight [16] algorithms need not maintain per-flow queues and do not require queue length information exchange. But max-weight need not be throughput optimal for multi-hop networks [17], and self-regulated max-weight also stabilizes the system only for a restricted class of arrivals [15]. In contrast, the virtual queue length based max-weight is throughput optimal [16] in multihop networks with fixed routing.

1.2. Our contribution

We consider a system with multiple users connected to a BS with an attached MEC server and cloud. The end users use multiple applications (see Fig. 1). The applications generate streams of services that are queued at the application layer of the users. The users can execute these services locally subject to their resource constraints or can offload these. The offloaded services are passed to the users’ MAC layers where they are placed in MAC queues. The users send these services to the BS through wireless links. The offloaded services from all the users are queued at the BS. The BS can execute these services at the attached MEC server subject to the server’s resource constraints or can further offload these to the cloud. MEC comes with new traffic characteristics and QoS requirements and warrants designing joint communication and compute resource allocation algorithms that minimize the costs associated with offloading, e.g., resource consumption at the MEC server and the delay in service execution at the cloud, while stabilizing all the application and MAC layer queues. These problems can be posed as joint routing and scheduling in multihop networks. However, communication, compute and storage resource allocations are activities at two different network layers, and application layer queue lengths are not available at MAC layer and vice versa. Hence, unlike [12,13] the classical back-pressure algorithm cannot be used. For instance, for uplink scheduling at the MAC layer, the back-pressure algorithm requires differences between the MAC queue lengths and the per service queue lengths at the MEC server, but the latter is available only at the application layer. We propose virtual queue length based task offloading and scheduling algorithms that do not require cross-layer message exchange. The proposed algorithms are throughput-optimal and also account for the task execution delays.

We note that our virtual queue based drift-plus-penalty approach is different from the ones in [6,11,18], where virtual queues are introduced to satisfy certain time average constraints other than stability of the queues. For example, [11,18], consider single hop networks with virtual queue to satisfy average cost and power constraints, respectively. These virtual queues can be introduced even if there are no actual queues to stabilize. We use virtual queues to ensure stability of the actual queues and virtual queue based drift-plus-penalty to achieve average delay arbitrarily close to optimal.

Our preliminary results appeared in [19]. Unlike [19], we now allow a service request to be forwarded to the cloud server in multiple slots, rather than in just one slot. The cloud starts serving the request only after it has been completely offloaded. We introduce a new throughput-optimal algorithm that yields smaller average queue lengths than the algorithms in [19]. Finally, we extend our framework to include local computation on users' devices within the MEC system.

Following is a preview of our main contributions.

- (a) We first consider an MEC system without local computation, where the users offload all their services to the BS. The services are either executed at the MEC server or are further offloaded to the cloud. We cast the joint queue stability and offloading cost minimization problem for the MEC system accounting for communication, storage, and computation constraints. We propose two joint scheduling and compute and storage resource allocation algorithms that do not require per-service queue length information at the MAC layer and cross-layer queue length information exchange as in the back-pressure based algorithms. Our algorithms use local virtual queue lengths and drift-plus-penalty framework and offer same performance as back-pressure based algorithms.
- (b) Our proofs are based on fluid limit techniques where the challenging part is to establish throughput optimality under drift plus penalty framework. Our analysis to obtain bounds on the average queue length and cost for processing services at the cloud is different from the existing works.
- (c) We validate our results via simulations and show the trade-off between the queuing and processing delays at the MEC server and the cost of offloading to the cloud. We also compare the proposed algorithms' performance to that of a back-pressure based algorithm via simulations.
- (d) We propose another throughput optimal algorithm by modifying the arrivals to the virtual queues. We demonstrate via simulations that it performs better than the previous algorithms in terms of the queue lengths.
- (e) We extend our framework to allow local computation at the users. We assume that BS's per service queue lengths are available at the users' application layers and can be used to compute "back-pressure". We propose compute resource allocation and offloading algorithms that use back-pressure at the users' application layers but only local queue length at the users' MAC layer and at the BS.

2. System model

We first consider an MEC system without local computation at the users. Our system consists of a set of users $\mathcal{U} = \{1, 2, \dots, U\}$ connected to a BS with an attached MEC server. The BS is also connected to the cloud through the Internet. The system evolves in discrete time, a unit of time being referred to as a *slot*.

2.1. Service request generation at the user

The users request services from the set $S = \{1, 2, \dots, S\}$. Let $A_{u,s}(t)$ be the number of type- s services requested by user u at the beginning of slot t . We assume that all the service arrival processes at all the users are i.i.d. with $E[A_{u,s}(t)] = \lambda_{u,s}$ and $E[A_{u,s}^2(t)] < \infty$. Hence, by the *strong law of large numbers (SLLN)*, $\lim_{t \rightarrow \infty} \frac{1}{t} \sum_{\tau=1}^t A_{u,s}(\tau) = \lambda_{u,s}$, with probability 1. We define $A_{\max} \triangleq \max_{s,u} A_{u,s}$. Let $d_s \in \mathbb{Z}_+$ be the number of packets generated at a user for each type- s request. We define $\min_s d_s \triangleq d_{\min}$ and $\max_s d_s \triangleq d_{\max}$. Let $A_u(t)$ be the number of packets at user u at slot t and $P_u(t)$ be the cumulative count until slot t . Then $A_u(t) = \sum_{s \in S} A_{u,s}(t) d_s$ and $P_u(t) = \sum_{\tau=1}^t A_u(\tau)$. Each user u maintains a *first-in-first-out (FIFO)* queue, $Q_u(t)$, where the packets are stored before being scheduled for transmission to the BS.

2.2. Wireless transmission to the BS

We consider a single wireless channel with J states; our analysis can easily be extended to multiple channels. We assume that the channel state process $j(t), t \geq 1$ is an irreducible, positive recurrent Markov chain with stationary distribution μ . Let $\eta_{j,u}$ be the rate achieved by user u when the channel is in state j .

2.3. Service request arrivals at the BS

We assume services are non-splittable [20], i.e., a task cannot be partially offloaded from the BS to the cloud. We further assume the MEC server can process a task only if all the data for that task arrives. Hence, the BS is said to have received a type- s service request once it receives all the d_s packets corresponding to that request. Let $V_{u,s}(t)$ be the number of type- s requests of user u that the BS receives at slot t . Since the packets arriving at the BS at slot t corresponding to service type- s may not be multiple of d_s , residual packets wait at the BS until all the packets of the corresponding service arrive. Let $c_{u,s}(t)$ be the residual packets of the type- s service from user u at the BS at slot t . Then $V_{u,s}(t) = \lfloor \frac{c_{u,s}(t) + D_{u,s}(t)}{d_s} \rfloor$ and $c_{u,s}(t) = c_{u,s}(t-1) + D_{u,s}(t-1) - V_{u,s}(t-1)d_s$. We note that, $c_{u,s} < d_s \forall s, u$. Let $A_{BS,s}(t)$ be the total number of type- s requests arrivals at slot t and $P_{BS,s}(t)$ be the cumulative count until slot t . Then $A_{BS,s}(t) = \sum_u V_{u,s}(t)$ and $P_{BS,s}(t) = \sum_{\tau=1}^t A_{BS,s}(\tau)$. The BS maintains a FIFO queue, $Q_{BS,s}(t)$, for each service s . We refer to $Q_{BS,s}(t)$ as per service queues at the MEC server. It can process the requests at the MEC server or forward them to the cloud.

2.4. Storage and computation

Let the MEC server have a CPU frequency C cycles/slot and storage R bits. If the data required for processing type- s requests is placed at the MEC server, we say that service- s is hosted by it. Let r_s denote the amount of data (in bits) required to be pre-stored at the MEC server to process type- s requests, and c_s be the number of CPU cycles needed to process a type- s request. We define the MEC server configuration at slot t , $X(t) = (X_s(t), s \in S)$, as follows.

$$X_s(t) = \begin{cases} 1, & \text{if service } s \text{ is hosted at the server at slot } t, \\ 0, & \text{otherwise.} \end{cases}$$

Clearly, $X(t) \in \mathcal{X}$ where

$$\mathcal{X} \triangleq \left\{ X \in \{0, 1\}^{|S|} : \sum_{s \in S} r_s X_s \leq R \right\}.$$

We use $N(t) = (N_s(t), s \in S)$ to denote the number of services scheduled at the MEC server at slot t . We require that $\sum_s N_s(t) c_s \leq C$. So, each type- s request needs exactly one slot for processing at the MEC server. Moreover, $N(t) \in \mathcal{N}(X(t))$ where

$$\mathcal{N}(X) \triangleq \left\{ N \in \mathbb{Z}_+^{|S|} : \sum_{s \in S} c_s N_s \leq C, N_s(1 - X_s) = 0 \right\}.$$

Let $\mathcal{N} \triangleq \cup_{X \in \mathcal{X}} \mathcal{N}(X)$. We can easily verify that

$$\mathcal{N} = \left\{ N \in \mathbb{Z}_+^{|S|} : \sum_{s \in S} c_s N_s \leq C, \sum_{s \in S} r_s \mathbb{1}_{\{N_s \geq 1\}} \leq R \right\}.$$

2.5. Service request offloading to the cloud

The link from the BS to the cloud has capacity C_{cloud} packets/slot. We use $E_s^c(t)$ and $\tilde{E}_s^c(t)$ to denote the number of type- s packets scheduled to be forwarded to the cloud by the BS at slot t and the actual number of forwarded type- s packets, respectively. Note that $\tilde{E}_s^c(t)$ can be strictly smaller than $E_s^c(t)$ if BS does not have enough type- s packets. Also, $E_s^c(t) \in \mathcal{E}^c$ where

$$\mathcal{E}^c \triangleq \left\{ E^c \in \mathbb{Z}_+^{|S|} : \sum_{s \in S} E_s^c \leq C_{cloud} \right\}.$$

2.6. Cost of offloading to the cloud

Let Δ_s be the fixed cost of offloading to the cloud for service type- s . Hence, we assume round trip time (RTT) incurred in sending a type- s request to the cloud, executing it there, and obtaining the result back. We define the total *cost of offloading to the cloud* at slot t as $Y(t) = \sum_{s \in S} \Delta_s \frac{\tilde{E}_s^c(t)}{d_s}$. We aim at minimizing

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[Y(t)]$$

subject to stability of the users' and the BS's queues as defined in Section 3.3. We note that the RTT is the sum of the queuing delays, the transmission delays, the propagation delays, and the processing delays at the intermediate routers (see [21, Section 1.4]). Among these, the queuing delays at the BS and intermediate routers depend on traffic intensity. Assuming a remote server across a WAN, the offloaded requests pass through many intermediate routers. Offloading more requests to the cloud would increase the queuing delay at the BS but will not substantially increase the aggregate traffic through the intermediate routers in the core network. Consequently, the queuing delays at these routers remain largely unaffected. Hence, we assume fixed RTT, Δ_s , independent of the offloading decisions.

3. Optimal scheduling

3.1. Capacity region

We first characterize the set of all service request arrival rates for which there exists an algorithm that stabilizes the system. Let $\mathcal{C}$ denote this set, also referred to as the capacity region of the MEC system. Following standard arguments (e.g., see [22]), it can be shown that

$$\mathcal{C} = \left\{ \lambda \in \mathbb{R}^{|S| \times |U|} : \sum_{s \in S} \lambda_{u,s} d_s \leq \sum_{j \in J} \mu_j \beta_{j,u} \eta_{j,u} \forall u, \right. \quad (1)$$

$$\left. \sum_{u \in U} \lambda_{u,s} \leq \sum_{N \in \mathcal{N}} \alpha_N N_s + \sum_{E^c \in \mathcal{E}^c} \phi(E^c) \frac{E_s^c}{d_s} \forall s, \right\} \quad (2)$$

Table 1
Notation.

Symbol	Description
$\mathcal{U}$	Set of users
$\mathcal{S}$	Set of services
$\mathcal{X}$	Set of service configurations MEC server can host
$\mathcal{N}(X)$	Set of number of service configurations MEC Server can process (given the service configuration X)
$\mathcal{E}^c$	Set of service configurations that can be sent to the cloud
$A_{u,s}(t)$	Number of s -type services requested by user u in slot t
$V_{u,s}(t)$	Number of s -type services requested by user u that arrives at the BS
$Y(t)$	Cost of offloading to the cloud
$A(t), \hat{A}(t)$	Arrival to the actual and virtual queues
$D(t), \hat{D}(t)$	Departure from the actual and virtual queues
$P(t), \hat{P}(t)$	Cumulative arrival to the actual and virtual queues
$Q(t), \hat{Q}(t)$	Actual and virtual queue lengths
Δ	Cost configuration of the services executed at the cloud
$E^c(t)$	Configuration of packets to be sent to the cloud

where

$$\sum_{u \in \mathcal{U}} \beta_{j,u} \leq 1, \beta_{j,u} \geq 0 \quad \forall j \in J \quad \forall u \in \mathcal{U}, \quad \sum_{N \in \mathcal{N}} \alpha_N \leq 1, \alpha_N \geq 0 \quad \forall N \in \mathcal{N},$$

$$\left. \sum_{E^c \in \mathcal{E}^c} \phi(E^c) \leq 1, \phi(E^c) \geq 0, \forall E^c \in \mathcal{E}^c \right\}.$$

The above characterization of the capacity region is similar to that for multi-hop networks in [16]. In particular, (1) ensures that the mean service rate of the packets of type- s tasks at the wireless link between user u and BS is no less than the mean arrival rate of the packets associated with type- s tasks at user u . Similarly, (2) ensures that the mean rate at which the type- s services are served at the MEC server or are offloaded to the cloud is no less than the mean type s task arrival rate at the BS.

In the following, with a slight abuse of notation, we use $Q_u(t)$ and $Q_{BS,s}(t)$ to denote the queues as well as the respective queue lengths. We summarize the notation in Table 1. Due to space constraint, we only provide outlines of the proofs which can be found in Appendix A.

3.2. Offloading and scheduling algorithm

Note that the system has $|\mathcal{U}|$ and $|\mathcal{S}|$ queues at the user and the server ends, respectively. For the proposed offloading and scheduling algorithms, we also maintain virtual queues corresponding to each of the $|\mathcal{U}| + |\mathcal{S}|$ queues. Let $\hat{Q}_u(t)$ and $\hat{Q}_{BS,s}(t)$ denote the lengths of the virtual queues. Further, we employ two tuneable parameters $\epsilon > 0$ and $W > 0$ that can be used to attain the desired tradeoff between the queuing and processing delays at the MEC server and the service processing delay at the cloud.

Algorithm 1 (Virtual Queue-based drift-plus-penalty)

At each user:

1. The arrival to virtual queue :

$$\hat{A}_u(t) = (1 + \epsilon) \frac{P_u(t)}{t}.$$

2. Scheduling policy at MAC layer:

$$u^*(t) \in \arg \max_u \hat{Q}_u(t) \eta_{j(t),u}$$

with ties broken arbitrarily.

3. Departure from virtual queue at user:

$$\hat{D}_u(t) = \begin{cases} \min\{\hat{Q}_u(t), \eta_{j(t),u}\}, & \text{if } u = u^*(t) \\ 0, & \text{otherwise.} \end{cases}$$

4. Departure from actual queue at user:

$$D_u(t) = \begin{cases} \min\{Q_u(t), \eta_{j(t),u}\}, & \text{if } u = u^*(t) \\ 0, & \text{otherwise.} \end{cases}$$

At the MEC server:

5. Arrival to the virtual queue:

$$\hat{A}_{BS,s}(t) = (1 + \epsilon) \frac{P_{BS,s}(t)}{t}.$$

6. Scheduling policy at Application layer: $N(t)$ and $E(t)$ are solutions to

$$\max_{N \in \mathcal{N}, E^c \in \mathcal{E}^c} \sum_{s \in \mathcal{S}} \hat{Q}_{BS,s}(t) N_s + \sum_{s \in \mathcal{S}} (\hat{Q}_{BS,s}(t) - W \Delta_s) \frac{E_s^c}{d_s}.$$

7. Departure from the virtual queue at the MEC server:

$$\hat{D}_{BS,s}(t) = \min \left\{ \hat{Q}_{BS,s}(t), N_s(t) + \frac{E_s^c(t)}{d_s} \right\}.$$

8. Departure from actual queue at the MEC server:

$$D_{BS,s}(t) = \tilde{N}_s(t) + \frac{\tilde{E}_s^c(t)}{d_s} \text{ where}$$

$$\tilde{N}_s(t) = \lfloor \min\{Q_{BS,s}(t), N_s(t)\} \rfloor \text{ to the MEC server}$$

$$\text{and } \tilde{E}_s^c(t) = \min \left(Q_{BS,s}(t) - \tilde{N}_s(t), \frac{E_s^c(t)}{d_s} \right) d_s \text{ to the cloud.}$$

The users' actual and virtual queue lengths evolve as follows:

$$Q_u(t+1) = Q_u(t) + A_u(t) - D_u(t), \quad (3)$$

$$\hat{Q}_u(t+1) = \hat{Q}_u(t) + \hat{A}_u(t) - \hat{D}_u(t). \quad (4)$$

Similarly, the MEC server's actual and virtual queue lengths evolve as follows:

$$Q_{BS,s}(t+1) = Q_{BS,s}(t) + A_{BS,s}(t) - D_{BS,s}(t), \quad (5)$$

$$\hat{Q}_{BS,s}(t+1) = \hat{Q}_{BS,s}(t) + \hat{A}_{BS,s}(t) - \hat{D}_{BS,s}(t). \quad (6)$$

Clearly, $D_{BS,s}(t) \leq N_s(t) + E_s^c(t) \forall s$. Let $\hat{P}_u(t)$ and $\hat{P}_{BS,s}(t)$ denote the cumulative arrivals to the virtual queues $\hat{Q}_u$ and $\hat{Q}_{BS,s}$, respectively, until slot t . We assume initial condition $A_{u,s}(0)=D_u(0)=D_{BS,s}(0)=\hat{D}_u(0)=\hat{D}_{BS,s}(0)=D_s^c(0)=0$.

Remark 1. We note that the per-service actual queues ($Q_{BS,s}(t)$) are rational, which may include fractions. While the MEC server executes a job within a slot, the packets sent to the cloud within a slot may not be multiples of the data sizes of each job. To illustrate this with an example, let us consider the queue length evolution of service type- s with a data size of 5 packets. Suppose the queue length at slot t is 10 services, arrival to that queue is 2 services, service by the MEC server is 5 services, and departure to the cloud is 7 packets. Then, in the next slot, the queue length will be $(10 + 2 - 5 - \frac{7}{5}) = 5.6$ services. Therefore, the queue length's fractional component is associated with the departure to the cloud. In the next slot the scheduled departure to the cloud will address the fractional part of the queue first, and the remaining part will be served in a FIFO manner.

3.3. Optimality results

Let $Q(t)=(Q_u(t), Q_{BS,s}(t))$ and $\hat{Q}(t)=(\hat{Q}_u(t), \hat{Q}_{BS,s}(t))$ denote the queue length vectors of the actual and the virtual queues at slot t .

Let $P(t)=(P_u(t), P_{BS,s}(t))$ be the vector of cumulative arrivals at the actual queues. Let us also define $\mathcal{Z}(t)=(Q(t), \hat{Q}(t), \frac{P(t)}{t+1})$.

$\mathcal{Z}(t), t \geq 0$ evolves as a Markov chain under Algorithm 1. We call this Markov chain stable if it is *positive Harris recurrent* [23, subsection 3]. Note that

$$\|\mathcal{Z}(t)\|_1 = \|Q(t)\|_1 + \|\hat{Q}(t)\|_1 + \frac{1}{t+1} \|P(t)\|_1.$$

We use $\mathcal{Z}^{(z)}(t), t \geq 0$ to refer to the Markov process with initial condition $\|\mathcal{Z}(0)\|_1 = z$. To analyze the stability of the Markov chain $\mathcal{Z}(t)$, we consider a sequence of scaled Markov chains $\frac{1}{z} \mathcal{Z}^{(z)}(zt), z=1, 2, \dots$. The following lemma characterizes positive Harris recurrence of $\mathcal{Z}(t)$ in terms of the scaled processes.

Lemma 1 ([16, Lemma 6]). *Let there be $\xi, T > 0$ such that for any sequence of processes $\frac{1}{z} \mathcal{Z}^{(z)}(zt), t \geq 0, z = 1, 2, \dots$, we have $\limsup_{z \rightarrow \infty} \mathbb{E} \left[\frac{1}{z} \|\mathcal{Z}^{(z)}(zt)\|_1 \right] \leq 1 - \xi$. Then the Markov chain $\mathcal{Z}^{(z)}(t), t \geq 0$ is stable.*

The following theorem shows that the scaled Markov chains $\frac{1}{z} \mathcal{Z}^{(z)}(zt), t \geq 0, z = 1, 2, \dots$ satisfy the condition for stability as in Lemma 1 under Algorithm 1.

Theorem 2. *The MEC System is stable under Algorithm 1 for any arrival rate vector λ such that $(1 + \epsilon)\lambda \in \mathcal{C}$ for $\epsilon > 0$.*

Proof (Outline). To prove stability of the MEC system, we first prove that the virtual queues are stable. Following Algorithm 1, after large enough time the arrival rates at the virtual queues become very close to $(1 + \epsilon)\lambda$. Hence, stability of the virtual queues implies that the service rates for the virtual queues are at least $(1 + \epsilon)\lambda$. Since the service rates of both the virtual and the actual queues are same, stability of the virtual queues implies stability of the actual queues.

Arrivals at the virtual queues at the users and at the MEC server constitute single-hop traffic. The packets in the virtual queues leave the network after service without getting transferred to the subsequent hop queues. Hence, the virtual queues decompose the multi-hop network into two single-hop networks. Therefore any exchange of queue lengths from the MEC server to the users is not required. Following are the main technical steps of the stability proof.

The process $\{Z(t), t \geq 0\}$ is scaled in both time and space by a sequence of integers $\{z_n\}$ such that $z_n \rightarrow \infty$ as $n \rightarrow \infty$ and the limit of the scaled processes $\{\frac{1}{z_n} Z^{(z_n)}(z_n t), t \geq 0\}$ is referred to as the fluid limit process, $z(t)$ [16,24].

(a) We show using SLLN that with probability 1

$$\limsup_{n \rightarrow \infty} \frac{1}{z_n} \left[\frac{1}{z_n t + 1} \|P^{(z_n)}(z_n t)\|_1 \right] = 0. \quad (7)$$

(b) We show that with probability 1 for any $\zeta > 0$ there exists a finite time T such that

$$\limsup_{n \rightarrow \infty} \frac{1}{z_n} \sum_u \hat{Q}_u^{(z_n)}(z_n t) < \zeta \forall t \geq T. \quad (8)$$

(c) We show that with probability 1 for any $\zeta_1 > 0$ there exists a finite time T_1 such that

$$\limsup_{n \rightarrow \infty} \frac{1}{z_n} \sum_s \hat{Q}_{BS,s}^{(z_n)}(z_n t) < \zeta_1 \forall t \geq T_1. \quad (9)$$

(d) Using the above results for virtual queues we show that there exists a finite time $T^* \geq \max\{T, T_1\}$ such that with probability 1

$$\limsup_{n \rightarrow \infty} \frac{1}{z_n} \left(\sum_u Q_u^{(z_n)}(z_n t) + \sum_s Q_{BS,s}^{(z_n)}(z_n t) \right) = 0 \forall t \geq T^*. \quad (10)$$

(e) We obtain Theorem 2 by combining the above results and by uniform integrability of the scaled sequence $\{\frac{1}{x} Z^{(x)}(xT^*), x = 1, 2, \dots\}$.

The proofs of Steps (a) and (d) are as in [16]. Step (b) also follows from a similar technique as in [24]. Our main contribution is the proof of Step (c). For an MEC system without cloud (i.e., when the scheduling policy at the MEC server is $\max_{X,N} \sum_s \hat{Q}_{BS,s}(t) N_s$), (9) follows from arguments similar to those in [16]. But we need more delicate analysis for the MEC system with cloud. We use the observation that the scaled limit of the cost $W \sum_s \frac{E_s^c(t)}{d_s} \Delta_s$ is 0 since W, Δ_s and $E_s^c(t)$ are all bounded. The detailed proof is in Appendix C. $\square$

3.3.1. Queue-length and cost bounds

Let $Y_{opt}(\lambda)$ be the minimum time average expected cost of offloading to the cloud that can be achieved by any control policy that stabilizes the system under the arrival rate λ . Let $(1 + \epsilon)\lambda \in C$. Then there exists a small enough number $\epsilon_1 > 0$ such that $(1 + \epsilon)\lambda + \underline{1}\epsilon_1 \in C$. Therefore there exists a stationary scheduling policy $\bar{\pi}$ such that [6]

$$\mathbb{E}[N_s^{\bar{\pi}}(t) + \frac{E_s^c \bar{\pi}(t)}{d_s}] \geq (1 + \epsilon) \sum_u \lambda_{u,s} + \epsilon_1 \forall s, \quad (11)$$

$$\mathbb{E}[Y^{\bar{\pi}}(t)] = Y_{opt}((1 + \epsilon)\lambda + \underline{1}\epsilon_1) \quad (12)$$

where $N_s^{\bar{\pi}}(t)$ and $E_s^c \bar{\pi}(t)$ are the numbers of type- s services served by the MEC server and packets sent to the cloud, respectively, and $Y^{\bar{\pi}}(t)$ is the cost of offloading to the cloud at slot t under policy $\bar{\pi}$.

Theorem 3. We obtain the following bounds for the average expected cost of offloading to the cloud and the average expected weighted virtual queue length at the MEC server.

$$\begin{aligned} \limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[Y(t)] &\leq \frac{B_2}{W} + Y_{opt}((1 + \epsilon)\lambda + \underline{1}\epsilon_1), \\ \limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[\sum_s \hat{Q}_{BS,s}(t) \sum_u \lambda_{u,s}] &\leq \frac{B_2}{\epsilon_1} + \frac{W Y_{opt}((1 + \epsilon)\lambda + \underline{1}\epsilon_1)}{\epsilon_1} \end{aligned}$$

where $B_2 = SU(1 + \epsilon)^2 A_{\max}^2 d_{\max}^2 + SC^2 + SC_{cloud}^2$.

Proof. Let us consider a Lyapunov function $V(t) = \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t)$ and define Lyapunov drift $\Delta V(t) = \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t+1) - \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t)$. Then,

$$\Delta V(t) + WY(t)$$

$$\begin{aligned}
&= \frac{1}{2} \sum_s \left(\left(\hat{Q}_{BS,s}(t) - N_s(t) - \frac{E_s^c(t)}{d_s} \right)^+ + \hat{A}_{BS,s}(t) \right)^2 - \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t) + WY(t) \\
&\leq \frac{1}{2} \sum_s \left(\hat{A}_{BS,s}^2(t) + N_s^2(t) + \left(\frac{E_s^c(t)}{d_s} \right)^2 \right) + \sum_s \hat{Q}_{BS,s}(t) \left(\hat{A}_{BS,s}(t) - N_s(t) - \frac{E_s^c(t)}{d_s} \right) + WY(t) \\
&\leq B_2 + \sum_s \hat{Q}_{BS,s}(t) \left((1+\epsilon) \frac{\sum_{\tau=1}^t A_s(\tau)}{t} \right) - \sum_s \hat{Q}_{BS,s}(t) \left(N_s(t) + \frac{E_s^c(t)}{d_s} \right) + WY(t)
\end{aligned} \tag{13}$$

where $A_s(t) = \sum_u A_{u,s}(t)$ and $B_2 = SU(1+\epsilon)^2 A_{\max}^2 d_{\max}^2 + SC^2 + SC_{cloud}^2$.

Let us define,

$$\Theta(T) \triangleq \sum_{t=1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \left(\frac{\sum_{\tau=1}^t A_s(\tau)}{t} - \sum_u \lambda_{u,s} \right) \right].$$

Adding and subtracting $(1+\epsilon) \sum_s \hat{Q}_{BS,s}(t) \sum_u \lambda_{u,s}$ from (13), and using (11) and (12) we obtain the following bounds.

Bound on the average expected cost:

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[Y(t)] \leq \frac{B_2}{W} + Y_{opt}((1+\epsilon)\lambda + \underline{1}\epsilon_1) + (1+\epsilon) \limsup_{T \rightarrow \infty} \frac{1}{T} \Theta(T). \tag{14}$$

Bound on the average expected weighted queue length:

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \sum_u \lambda_{u,s} \right] \leq \frac{B_2}{\epsilon_1} + \frac{WY_{opt}((1+\epsilon)\lambda + \underline{1}\epsilon_1)}{\epsilon_1} + (1+\epsilon) \limsup_{T \rightarrow \infty} \frac{1}{T} \Theta(T). \tag{15}$$

Both the bounds follow using standard arguments as in [6, Chapter 3]. Their proofs can be found in Appendix D. The nonstandard part is proving that $\limsup_{T \rightarrow \infty} \frac{1}{T} \Theta(T) = 0$. Below, we provide the key arguments of this proof.

Firstly, Harris positive recurrence of the Markov chain $\mathcal{Z}(t)$ implies that $\exists$ a petite set $B = \{z \in \mathcal{Z} : \|z\|_1 \leq \kappa\}$ for some $\kappa > 0$ and $\exists$ a $T > 0$ such that $\sup_{z \in B} \mathbb{E}_z[\tau_B(T)] < \infty$, where $\tau_B(T) = \inf\{t \geq T : \mathcal{Z}(t) \in B\}$ [23, Theorem 3.1]. This further implies that $\sum_s \hat{Q}_{BS,s}(t) \leq \kappa + \tau_B(T)(1+\epsilon)A_{\max}$ and $\mathbb{E}[\sum_s \hat{Q}_{BS,s}(t)] \leq \kappa + \mathbb{E}[\tau_B(T)](1+\epsilon)A_{\max} < \kappa_1$ for some $\kappa_1 > 0$. Secondly, since the arrivals satisfy SLLN, for a given $\delta > 0$ $\exists$ a T_δ such that $\left| \frac{\sum_{\tau=1}^t A_s(\tau)}{t} - \sum_u \lambda_{u,s} \right| \leq \delta \forall t \geq T_\delta$ with probability 1.

Now, we divide $\Theta(T)$ into two part summation as follows:

$$\Theta(T) = \sum_{t=1}^{T_\delta} \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \left(\frac{\sum_{\tau=1}^t A_s(\tau)}{t} - \sum_u \lambda_{u,s} \right) \right] + \sum_{t=T_\delta+1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \left(\frac{\sum_{\tau=1}^t A_s(\tau)}{t} - \sum_u \lambda_{u,s} \right) \right]. \tag{16}$$

The first term is finite since T_δ is finite and the second term could be made arbitrarily small by choosing a small enough δ . Then taking $T \rightarrow \infty$ we obtain $\limsup_{T \rightarrow \infty} \frac{1}{T} \Theta(T) = 0$. $\square$

By choosing $\epsilon_1 > 0$ arbitrarily small we can obtain static policies that stabilize the MEC system and yield a cost arbitrarily close to $Y_{opt}((1+\epsilon)\lambda)$. It is clear from Theorem 3 that increasing W decreases the upper bound on the average expected cost of offloading to the cloud and increases the average expected virtual queue length. This could be understood as follows. Algorithm 1 chooses the schedules at the MEC server based on the optimization

$$\max_{N \in \mathcal{N}, E^c \in \mathcal{E}^c} \sum_{s \in S} \hat{Q}_{BS,s}(t) N_s + \sum_{s \in S} (\hat{Q}_{BS,s}(t) - W \Delta_s) \frac{E_s^c}{d_s}$$

which can be decomposed into two parts:

$$N_s(t) \in \arg \max_{N \in \mathcal{N}} \sum_{s \in S} \hat{Q}_{BS,s}(t) N_s, \tag{17}$$

$$E^c(t) \in \arg \max_{E^c \in \mathcal{E}^c} \sum_{s \in S} (\hat{Q}_{BS,s}(t) - W \Delta_s) \frac{E_s^c}{d_s}. \tag{18}$$

$E_s^c(t)$ will be positive only when $\hat{Q}_{BS,s}(t)$ exceeds $W \Delta_s$; in other words, BS will send the services to the cloud if the virtual queue length exceeds the threshold of $W \Delta_s$. For a given arrival rate, higher value of W implies less number of services being sent to the cloud. Therefore, the service rates of the virtual queues decrease. If the arrival rates are close to the capacity region boundary, a higher value of W implies higher virtual queue lengths. Since the service rates of the virtual and the actual queues are same as per Algorithm 1, the queue lengths of the actual queues also increase with W . By Little's Law the average delay is proportional to the average queue lengths. So, the parameter W facilitates a trade-off between the queuing and processing delays at the MEC server and the cost of offloading to the cloud.

Remark 2. While our fluid limit based analysis is similar to the one in Ji et al. [16], our analysis differs from [16] in the following aspects also:

- (a) In addition to proving the stability of the queues, we also obtain an upper bound on the average cost of offloading to the cloud and by tuning the parameter W we can achieve the delay cost arbitrary close to optimal. To obtain this result we use the stability of the queues in a novel way.
- (b) Unlike [16], we do not assume a deterministic network with fixed link rates, we consider a stochastic wireless network with time varying channels.
- (c) We allow the queues at the MEC server, $Q_{BS,s}(t)$ to be fractional to capture the departure to the cloud in terms of packets while in [16] all the actual queues are integer-valued. To be precise, these fractional queues are lower bounds on the real queue lengths at the MEC server. The stability of the MEC servers' queues is ensured thanks to the infinite capacity of the cloud.

3.4. An alternative algorithm

Here, we propose another algorithm where the scheduler solves the same optimization problems, but the services sent to the cloud differ from Algorithm 1.

Algorithm 2 (Virtual queue-based drift-plus-penalty with controlled departure). This Algorithm is same as Algorithm 1 except 7 and 8 are replaced by 7* and 8*.

(7*) Departure from the virtual queue at the MEC server:

$$\begin{aligned}\hat{D}_{BS,s}(t) &= \hat{N}_s(t) + \frac{\hat{E}_s^c(t)}{d_s} \text{ where} \\ \hat{N}_s(t) &= \min\{Q_{BS,s}(t), N_s(t)\} \text{ and} \\ \hat{E}_s^c(t) &= (\min\{(\hat{Q}_{BS,s}(t) - \hat{N}_s(t) - W\Delta_s)^+, \frac{E_s^c(t)}{d_s}\})d_s.\end{aligned}$$

(8*) Departure from actual queue at the MEC server:

$$\begin{aligned}D_{BS,s}(t) &= \tilde{D}_s^c(t) + \tilde{N}_s(t) \text{ where} \\ \tilde{N}_s(t) &= \lfloor \min\{Q_{BS,s}(t), N_s(t)\} \rfloor \text{ to the MEC server} \\ \text{and } \tilde{E}_s^c(t) &= \left(\min \left\{ (Q_{BS,s}(t) - \tilde{N}_s(t) - W\Delta_s)^+, \frac{E_s^c(t)}{d_s} \right\} \right) d_s \text{ to the cloud.}\end{aligned}$$

Clearly, the difference between Algorithm 1 and Algorithm 2 lies only in the departures from the virtual and the actual queues at the MEC server.

Theorem 4. *The MEC Network is stable under Algorithm 2 for any arrival rate vector λ such that $(1 + \epsilon)\lambda \in C$ for $\epsilon > 0$. Moreover, Algorithm 2 offers the same queue-length and cost bounds as in Theorem 3 except B_2 being replaced with $B_2 + S(C + W\Delta_{\max} + C_{cloud})^2$.*

Proof. The proof of this theorem is along the same lines as the proofs of Theorems 2 and 3. However, there is a difference in the constant of the bounds provided in Theorem 3. See Appendix E for the detailed proof. $\square$

Complexity of the scheduling algorithms: The optimization problem at the MEC server under Algorithm 1 is NP-Hard. Let us consider the first part of the problem (i.e., (17)). Recall that $\mathcal{N} \triangleq \cup_{X \in \mathcal{X}} \mathcal{N}(X)$. So, the optimization in (17) can be decomposed into (a) optimizing over $N \in \mathcal{N}(X)$ for a given X and (b) optimizing over all $X \in \mathcal{X}$. The subproblem in (a) can be expressed as

$$\begin{aligned}P1 : \quad & \max \sum_{s \in I_X} \hat{Q}_{BS,s}(t) N_s \\ \text{subject to} \quad & \sum_{s \in I_X} N_s c_s \leq C \\ & N_s \in [N_{\max}] \quad \forall s \in I_X\end{aligned}$$

where $I_X = \{s : X_s \neq 0\}$. $P1$ is a bounded Knapsack problem, a known NP-Hard problem [25]. This can be solved using Dynamic Programming (DP) [25]. Further, the set $\mathcal{X}$ can be obtained using the brute-force search method. As the number of services increases, the complexity of finding the set $\mathcal{X}$ increases exponentially. We leave the development of a low complexity algorithm as our future work. However, (17) can be approximately solved by the approach in [8]. The second part of the problem (i.e., (18)) has the same structure as using $P1$ and can also be solved by DP. Since the optimization problem at the MEC server is same for both, Algorithms 1 and 2, Algorithm 2 is also NP-Hard. We believe that our “maxweight” algorithms can be altered as the appropriate “pick and compare” [26] or “power of two choices” [27] algorithms to tackle NP Hardness without affecting the stability claims. However, to the best of our knowledge a virtual queue and fluid limits based analysis of “pick and compare” or “power of two choices” algorithms is not available. This is an intriguing topic for future research.

3.5. Choice of ϵ

Implementation of the above algorithms requires a suitable choice of ϵ such that $(1+\epsilon)\lambda \in \mathcal{C}$. Since the arrival rates are unknown, it warrants choosing sufficiently small ϵ . However, a small ϵ leads to higher actual queue lengths than for a larger and still conformant ϵ . To avoid this problem, we present another algorithm as follows:

Algorithm 3 (Virtual queue-based drift-plus-penalty with virtual queue dependent arrival). This algorithm is same as Algorithm 1 except the ϵ in 1 and 5 are replaced by $\max\left\{\frac{1}{Q_u(t)+1}, \epsilon\right\}$ and $\max\left\{\frac{1}{Q_{BS,s}(t)+1}, \epsilon\right\}$, respectively.

The essence of this algorithm is to increase and decrease the arrivals to the virtual queues when their queue lengths are small and large, respectively. Obviously, the instantaneous arrival to a virtual queue ($\hat{Q}_u(t)$ or $\hat{Q}_{BS,s}(t)$) is more than $(1+\epsilon)$ times the running average arrival rate to that queue. Increasing the arrivals to the virtual queues increases the service rates of the virtual queues, provided that the virtual queues are stable. Hence, service rates of the actual queues also increase as the scheduled service is same for the virtual and actual queues.

The following theorem shows that the MEC system is stable and obtains bounds on average cost and queue length.

Theorem 5. *The MEC Network is stable under Algorithm 3 for any arrival rate vector λ such that $(1+\epsilon)\lambda \in \mathcal{C}$ for $\epsilon > 0$. Moreover, Algorithm 3 offers the same queue-length and cost bounds as in Theorem 3 except B_2 being replaced with $B_2 + SU_{A_{\max}}$.*

Proof. The proof of this theorem is along the same lines as the proofs of Theorems 2 and 3. $\square$

This idea is motivated by the algorithm proposed by Liu et al. [15], where they replaced ϵ by $\frac{1}{\hat{Q}_u(t)}$ or $\frac{1}{\hat{Q}_{BS,s}(t)}$, as applicable. Under this modification, Liu et al. [15] showed stability of the queues for any arrival rates $\lambda \in \mathcal{C}$. However, they imposed a stronger “regularity” condition on arrivals than satisfying SLLN. Under their regularity condition, all the sample paths of the arrivals follow uniform convergence while in SLLN arrivals converge to the mean arrival rate almost surely. To the best of our knowledge, establishing stability in the absence of this regularity condition on the arrivals is an open problem.

Remark 3. Our policies, i.e., Algorithms 1, 2 and 3 are throughput optimal and yield delays that are only $O(1/W)$ away from the optimal delays for offloading to the cloud, where W is a tuneable parameter. We also derive several insights from the theoretical performance bounds as follows:

1. By increasing W , the delay for offloading to the cloud can be reduced at the cost of increased queue lengths at the MEC server. Algorithm 2 uses more controlled departure to the cloud than Algorithm 1. We further compare the performances of Algorithms 1 and 2 in Section 6.2.2.
2. We also use another tuneable parameter, ϵ , for the trade-off between the actual and the virtual queue lengths. Since increasing ϵ increases the number of arrivals to the virtual queues, the stability of virtual queues guarantees an increase in service rates due to an increase in ϵ . Since the service rates of the actual queues are the same as the service rate of the corresponding virtual queues, increasing ϵ yields smaller actual queue lengths at the cost of large virtual queue lengths. To ensure the stability of the virtual queues (and thereby, of the actual queues), ϵ must be smaller than an arrival rate dependent threshold. Since the arrival rates are unknown, it warrants choosing conservatively small ϵ . Algorithm 3 improvises Algorithms 1 and 2 to offer small actual queue lengths for small ϵ . We further compare the performances of Algorithms 1 and 3 in Section 6.5.

4. Back-pressure scheduling

To benchmark the performance of Algorithms 1 and 2, we also develop a back-pressure (BP) based scheduling algorithm which requires per-flow queue length information at each hop. As opposed to the model in Section 2 where each user maintains only one queue, here each user has S queues, one per service type. Let $Q_{u,s}(t)$ be the queue length for the type- s services at user u . Let $A_{u,s}(t)$ and $D_{u,s}(t)$ be the type- s service arrivals at user u and packet departures from $Q_{u,s}(t)$, respectively, at slot t . Then $Q_{u,s}(t+1) = Q_{u,s}(t) + A_{u,s}(t) - D_{u,s}(t)$. The MEC server maintains per-service queues $Q_{BS,s}(t)$ as before.

BP-based drift-plus-penalty algorithm

At each user:

1. Scheduling policy at MAC layer:
 $w_u(t) = \max_s(Q_{u,s}(t) - d_s Q_{BS,s}(t))^+$,
 $u^*(t) \in \arg \max_u w_u(t) \eta_{j(t),u}$, and
 $s^*(t) \in \arg \max_s (Q_{u^*(t),s}(t) - d_s Q_{BS,s}(t))^+$.
2. Departure from user:
 $D_{u,s}(t) = \begin{cases} \min\{Q_{u,s}(t), \eta_{j(t),u}\}, & \text{if } u = u^*(t), s = s^*(t) \\ 0, & \text{otherwise.} \end{cases}$

At MEC server:

1. Scheduling policy at application layer: $N(t)$ and $E(t)$ are solutions to

$$\max_{N \in \mathcal{N}, E^c \in \mathcal{E}^c} \sum_{s \in S} Q_{BS,s}(t) d_s^2 N_s + \sum_{s \in S} (Q_{BS,s}(t) d_s^2 - W \Delta_s) \frac{E_s^c}{d_s}.$$

2. Departure from actual queue at the MEC server:

$$D_{BS,s}(t) = \tilde{N}_s(t) + \frac{\tilde{E}_s^c(t)}{d_s} \text{ where} \\ \tilde{N}_s(t) = \lfloor \min\{Q_{BS,s}(t), N_s(t)\} \rfloor \text{ to the MEC server} \\ \text{and } \tilde{E}_s^c(t) = \min\left(Q_{BS,s}(t) - \tilde{N}_s(t), \frac{E_s^c(t)}{d_s}\right) d_s \text{ to the cloud.}$$

Recall that in Section 2.3, we introduced the residual packets, $c_{u,s}(t)$ for type- s service from user u at the BS at slot t . Where $A_{BS,s}(t) = \sum_u \lfloor \frac{c_{u,s}(t) + D_{u,s}(t)}{d_s} \rfloor$ and $c_{u,s}(t) = c_{u,s}(t-1) + D_{u,s}(t-1) - \lfloor \frac{c_{u,s}(t-1) + D_{u,s}(t-1)}{d_s} \rfloor d_s$.

Let $\Gamma(t) = (Q_{u,s}(t), Q_{BS,s}(t))$. It is easy to see that $\Gamma(t), t \geq 0$ evolves as an irreducible Markov Chain [28] under BP based drift-plus-penalty algorithm. The following theorem states the Markov chain $\Gamma(t), t \geq 0$ is irreducible and positive recurrent thereby implying that the MEC system is stable under BP based drift-plus-penalty algorithm.

Theorem 6. *The MEC system is stable under BP based drift-plus-penalty algorithm for any arrival rate λ such that $(1+\epsilon)\lambda \in C$ for any $\epsilon > 0$.*

Proof (Outline). We consider the Lyapunov function $L(\Gamma(t)) = \sum_s \sum_u Q_{s,u}^2(t) + \sum_s Q_{BS,s}^2(t) d_s^2$. We obtain T -step Lyapunov drift using the following conditional expectation:

$$E \left[V((n+1)T) - V(nT) + \sum_{\tau=0}^{T-1} WY(nT + \tau) | \mathbf{Q}(nT) \right]. \quad (19)$$

The cumulative arrival to BS can be expressed as:

$$\sum_{\tau=0}^{T-1} A_{BS,s}(nT + \tau) \leq \sum_u \left(\frac{\sum_{\tau=0}^{T-1} D_{u,s}(nT + \tau)}{d_s} + \frac{c_{u,s}(nT)}{d_s} \right). \quad (20)$$

Considering (20), we obtain

$$E \left[V((n+1)T) - V(nT) + \sum_{\tau=0}^{T-1} WY(nT + \tau) | \mathbf{Q}(nT) \right] \\ \leq SUTK_1 + STK_2 - TE \left[\sum_u \sum_s Q_{u,s}(nT) \lambda_{u,s} d_s (\epsilon' - \frac{c_{u,s}(nT)}{Td_s}) | \mathbf{Q}(nT) \right] + T^2 K_3 + \frac{T(T-1)}{2} SU(\eta_{\max} + \eta_{\max} + U d_{\max} + A_{\max}^2 d_{\max}^2) \\ - TE \left[\sum_s Q_{BS,s}(nT) d_s^2 (\epsilon - \epsilon') \sum_u \lambda_{u,s} | \mathbf{Q}(nT) \right] + T(T-1) SU(C + C_{cloud})(1+\epsilon) A_{\max} d_{\max}^2 + T \frac{WY_{opt}((1+\epsilon)\lambda + \frac{1}{T\epsilon_1})}{\epsilon_1} \quad (21)$$

where $0 < \epsilon' < \epsilon$ such that $(1+\epsilon)\lambda \in C$, $K_1 = A_{\max}^2 d_{\max}^2 + \eta_{\max}^2$, $K_2 = d_{\max}^2 \left(\frac{\eta_{\max}}{d_s} + U + C + C_{cloud} + 2(A_{\max} + C + C_{cloud})^2 \right)$, and $K_3 = (\eta_{\max} + U)^2$. Since $\sum_u \frac{c_{u,s}(nT)}{d_s} \leq U$, there exists an integer $T_{\epsilon'}$ such that $\frac{U}{T_{\epsilon'}} < \epsilon'$ and we further define $\hat{\epsilon} \triangleq \epsilon' - \frac{U}{T_{\epsilon'}}$. Therefore, we obtain negative drift for sufficiently large queue lengths. $\square$

Theorem 7. *We obtain the following bounds for the average expected cost of offloading to the cloud and the average expected weighted queue length at the MEC server.*

$$\lim_{N \rightarrow \infty} \frac{1}{NT_{\epsilon'}} \sum_{n=0}^{N-1} E \left[\hat{\epsilon} \sum_u \sum_s Q_{u,s}(nT_{\epsilon'}) \lambda_{u,s} d_s + \sum_s Q_{BS,s}(nT_{\epsilon'}) d_s^2 (\epsilon - \epsilon') \sum_u \lambda_{u,s} \right] \\ \leq K_4 + \frac{WY_{opt}((1+\epsilon)\lambda + \frac{1}{T\epsilon_1})}{\epsilon_1}, \quad (22)$$

$$\lim_{N \rightarrow \infty} \frac{1}{NT_{\epsilon'}} \sum_{n=0}^{N-1} \sum_{\tau=0}^{T_{\epsilon'}-1} E[Y(nT_{\epsilon'} + \tau)] \leq \frac{K_4}{W} + \frac{Y_{opt}((1+\epsilon)\lambda + \frac{1}{T\epsilon_1})}{\epsilon_1} \quad (23)$$

where $K_4 = SUTK_1 + STK_2 + T_{\epsilon'} K_3 + \frac{(T_{\epsilon'}-1)}{2} SU(\eta_{\max} + U d_{\max} + A_{\max}^2 d_{\max}^2) + (T_{\epsilon'} - 1) SU(C + C_{cloud})(1+\epsilon) A_{\max} d_{\max}^2$ and K_1, K_2, K_3 and $T_{\epsilon'}$ are as in Theorem 6.

Proof. We obtain the bounds of Theorem 7 by rearranging the terms of (21), dividing by $NT_{\epsilon'}$, and taking limits. $\square$

Remark 4. If $d_s = 1$ for all s (i.e., each service has unit data size), the above algorithm is exactly same as BP-based scheduling for multihop communication networks.

5. MEC system with local computation

We now extend our analysis to incorporate local computation at the users' devices in addition to computation at the MEC server and at the cloud. As in Section 2.3, the services are non-splittable, i.e., a task cannot be partially offloaded from the user to the BS or from the BS to the cloud.

5.1. System model

We assume that each user $u \in \mathcal{U}$ has a CPU of frequency C_u cycles/slot. We consider the same service arrival process as in Section 2.1. Each user maintains $|S|$ queues at the application layer and a single queue at the MAC layer; the application layer queues store services in the unit of tasks whereas the MAC layer queues store them as packets. We denote the application and MAC layer queues by $Q_{u,s}(t)$ and $Q_u(t)$, respectively. Let $A_{u,s}(t) \in \mathbb{Z}_+$ be the number of tasks arriving at the queue $Q_{u,s}(t)$ and $A_{u,s}^{MAC}(t) \in \mathbb{Z}_+$ be the number of type- s tasks sent to the MAC layer queue $Q_u(t)$ at slot t . Then, arrival to the queue $Q_u(t)$ at slot t is $\sum_s A_{u,s}^{MAC}(t)d_s$. Let $n_{u,s}(t) \in \mathbb{Z}_+$ be the number of services locally executed at the user. We denote $n_u(t) = (n_{u,s}(t), s \in S)$. At any slot t , $n_u(t) \in \mathcal{N}_u$ where

$$\mathcal{N}_u = \left\{ n_u \in \mathbb{Z}_+^{|S|} : \sum_s n_{u,s} c_s \leq C_u \right\}.$$

5.1.1. Cost of offloading the services to the BS

We assume unit offloading cost per unit computation, i.e., the offloading cost for each offloaded type- s task is c_s . Since the services sent to the MAC queues are eventually offloaded to the BS, we consider $\sum_s c_s A_{u,s}^{MAC}(t)$ as the cost of offloading services at slot t . We assume that wireless transmission to the BS, service arrivals at the BS, storage, and computation at the BS, and offloading to the cloud are the same as in Sections 2.2, 2.3, 2.4, and 2.5, respectively.

5.2. Problem formulation

We aim to minimize the sum of cost of offloading to the BS and cloud. In particular, we aim to minimize

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E} \left[\sum_s c_s A_{u,s}^{MAC}(t) + \sum_s \Delta_s \frac{\bar{E}_s^c(t)}{d_s} \right]$$

subject to stability of the users' and the MEC server's queues as defined in Section 3.3.

Below we express the capacity region of the MEC system under consideration before presenting the communication and compute resource allocation algorithms in Section 5.3.

Remark 5. We note that our model does not consider any latency constraints but we aim to minimize latency. We account for latencies in our framework through the following means.

1. The cost of offloading a task to cloud captures the delay cost of sending the task to cloud and retrieving the task results. It also depends on the task's type and is denoted by Δ_s .
2. The queuing delays at the MAC layers of the users and at the application layer of the MEC server are accounted for in terms of the respective queue lengths. Recall that Little's law implies that reducing the average queue lengths reduces the associated average queuing delays also.

5.2.1. Capacity region

Let Λ be the capacity region for this set up. We assume no constraints on the number of services that a user's application layer can send to its MAC layer at any slot. Following [29, Lemma 8.3], an arrival rate vector $\lambda \in \Lambda$ if there exists a $f_{u,s} \geq 0$ for all s, u such that

$$\lambda_{u,s} \leq \sum_{n_u \in \mathcal{N}_u} \bar{\pi}(n_u) n_{u,s} + f_{u,s} \quad \forall s, u \quad (24)$$

$$\sum_{s \in S} f_{u,s} d_s \leq \sum_{j \in J} \mu_j \beta_{j,u} n_{j,u} \quad \forall u, \quad (25)$$

$$\sum_{u \in \mathcal{U}} f_{u,s} \leq \sum_{N \in \mathcal{N}} \alpha_N N_s + \sum_{E^c \in \mathcal{E}^c} \phi(E^c) \frac{E_s^c}{d_s} \quad \forall s, \quad (26)$$

where

$$\sum_{n_u \in \mathcal{N}_u} \bar{\pi}(n_u) \leq 1, \quad \sum_{u \in \mathcal{U}} \beta_{j,u} \leq 1, \quad \beta_{j,u} \geq 0 \quad \forall j \in J \quad \forall u \in \mathcal{U},$$

$$\sum_{N \in \mathcal{N}} \alpha_N \leq 1, \alpha_N \geq 0 \forall N \in \mathcal{N},$$

$$\sum_{E^c \in \mathcal{E}^c} \phi(E^c) \leq 1, \phi(E^c) \geq 0, \forall E^c \in \mathcal{E}^c.$$

In the above inequalities, $f_{u,s}$ denotes the rate at which type- s services are sent from user u 's application layer to its MAC layer. Inequality (24) asserts that the sum of the mean local computation rate of type- s services at user u and the rate at which type- s services are sent from user u 's application layer to its MAC layer is no less than the mean type s service arrival rate at user u . Inequalities (25) and (26) have similar rationale as (1) and (2), respectively.

5.3. Scheduling

The MEC system with local computation is similar to a three-hop network. Routing of the services at the users' application layers is not fixed, unlike the MEC system, which does not use local computation. The virtual queue-based algorithm (without back-pressure) applies to networks with fixed routing [16], and hence cannot be applied to the present problem. In stead we exploit the availability of the MEC server's queue lengths at the users' application layers and propose a back-pressure based algorithm for compute resource allocation and offloading at the users. Under the back-pressure algorithm, each user controls the number of services to serve locally and to offload by comparing its application layer queues and the MEC server's queues for each service type.

Next, we propose our algorithm, where $W, W_1 > 0$ are two tuning parameters.

Algorithm 4

At each user:

1. Scheduling service for local computation at Application layer: $n_{u,s}(t)$ is the solution to

$$\max_{n_u \in \mathcal{N}_u} \sum_{s \in S} Q_{u,s}(t) n_{u,s}. \quad (27)$$

Services executed locally:

$$\tilde{n}_{u,s}(t) = \min \{Q_{u,s}(t), n_{u,s}(t)\}.$$

2. Departure from application to MAC layer:

$$A_{u,s}^{MAC}(t) = \begin{cases} Q_{u,s}(t) - \tilde{n}_{u,s}(t), & \text{if } (Q_{u,s}(t) - Q_{BS,s}(t) - W_1 c_s) > 0 \\ 0, & \text{otherwise.} \end{cases} \quad (28)$$

3. Departure from user:

$$D_u^{MAC}(t) = \begin{cases} \min\{Q_u(t), \eta_{j,u}(t)\}, & \text{if } u = u^*(t) \\ 0, & \text{otherwise.} \end{cases}$$

where $u^*(t) \in \arg \max_u Q_u(t) \eta_{j(u),u}$.

At MEC server:

1. Scheduling policy at application layer: $N(t)$ and $E(t)$ are solutions to

$$\max_{N \in \mathcal{N}, E^c \in \mathcal{E}^c} \sum_{s \in S} Q_{BS,s}(t) N_s + \sum_{s \in S} (Q_{BS,s}(t) - W \Delta_s) \frac{E_s^c}{d_s}.$$

2. Departure from queue at the MEC server:

$$D_{BS,s}(t) = \tilde{N}_s(t) + \frac{\tilde{E}_s^c(t)}{d_s} \text{ where}$$

$$\tilde{N}_s(t) = \lfloor \min\{Q_{BS,s}(t), N_s(t)\} \rfloor \text{ to the MEC server}$$

$$\text{and } \tilde{E}_s^c(t) = \min \left(Q_{BS,s}(t) - \tilde{N}_s(t), \frac{E_s^c(t)}{d_s} \right) d_s \text{ to the cloud.}$$

The users' queue lengths evolve as at application layer:

$$Q_{u,s}(t+1) = Q_{u,s}(t) + A_{u,s}(t) - A_{u,s}^{MAC}(t) - \tilde{n}_{u,s}(t). \quad (29)$$

At MAC layer:

$$Q_u(t+1) = Q_u(t) + \sum_s A_{u,s}^{MAC}(t) d_s - D_u^{MAC}(t). \quad (30)$$

Table 2
Service specifications.

Services	Data size (MB)	Storage (GB)	CPU cycles ($\times 10^9$)	Cost of offloading to the cloud
Face recognition	10	10	2.5	3
Augmented reality	10	12	3	3
Mobile cloud gaming	8	16	3.5	2.4
Wearable sensor	1.5	10	1	1
Video editing	9	2	3	6

Similarly, the MEC server's queue lengths evolve as

$$Q_{BS,s}(t+1) = Q_{BS,s}(t) + A_{BS,s}(t) - D_{BS,s}(t). \quad (31)$$

Remark 6. At application layer of each user, we solve the following optimization problem:

$$\max_{n_u \in \mathcal{N}_u} \sum_{s \in S} Q_{u,s}(t) n_{u,s} + \max_{s \in S} (Q_{u,s}(t) - Q_{BS,s}(t) - W_1 c_s) a_{u,s}^{MAC}. \quad (32)$$

Since there are no constraints on the number of services that can be sent from the application to the MAC layer, whenever $(Q_{u,s}(t) - Q_{BS,s}(t) - W_1 c_s)$ is positive for some service type- s , the solution to the optimization problem (32) $a_{u,s}^{MAC}$ is ∞ . This further implies our proposed algorithm sends all the services to the MAC remaining in the queue after local computation. Hence, the departure to the MAC layer is as (28).

Remark 7.

1. Our proposed algorithm needs queue length information of the MEC server's queues at the application layer queues of users for scheduling. Since the MEC server's queues are at the application layer, we do not need cross-layer information exchange, unlike back-pressure-based scheduling algorithms.
2. In Algorithm 4, by tuning W and W_1 , we can control the queue lengths at the users' application and MAC layers and at the MEC server. Increasing W increases the queue length at the MEC server and reduces the delay for sending the jobs to the cloud, thus offering a trade-off between the delays at the MEC server and for offloading to the cloud. Furthermore, increasing W_1 reduces the number of jobs sent to the MAC layer of the users and hence increases the application layer queue lengths. Increasing W_1 can increase or reduce MAC layer queue lengths depending on the variation of the MEC server queue lengths. We discuss these properties in detail in Sections 6.6.2 and 6.6.3.

We demonstrate via simulations that our proposed algorithm stabilizes the queues.

6. Numerical results

In this section, we first consider an MEC system without local computation at the users. Later, we consider an MEC system with local computation in Section 6.6. We evaluate the performance of Algorithm 1 via simulations. Then, we compare the queue lengths and cost of offloading to the cloud of Algorithm 1 and Algorithm 2. Furthermore, we show that Algorithms 1 and 2 offer almost the same performance as BP-based Algorithm. Lastly, we compare the queue length performance of Algorithm 3 and Algorithm 1. We consider 5 different services and 20 users associated with a BS. We study two performance metrics, the aggregate queue lengths (i.e., the sum of queue lengths of 20 users at the users and the sum of queue lengths of 5 services at the MEC server) and the average cost of offloading to the cloud. The MEC server has a storage capacity of 25 GB and CPU of frequency 5 GHz [8], and the cloud has a transmission rate $C_{cloud} = 100$ Mbps [13,30]. The service specifications are as in [8,31,32] and are listed in Table 2.

We consider each slot length is 1s, each packet size is 1 Mb [33], and $\epsilon = 10^{-4}$, and the transmission channel between is the Gilbert–Elliot channel with transition probability matrix $\begin{bmatrix} 0.6 & 0.4 \\ 0.5 & 0.5 \end{bmatrix}$. The uplink rates for two different channel states are 199 and 163 Mbps [8]. We assume that the arrival process is i.i.d. over the slots and in each slot number of arrivals is a Poisson random variable with a given arrival rate vector.

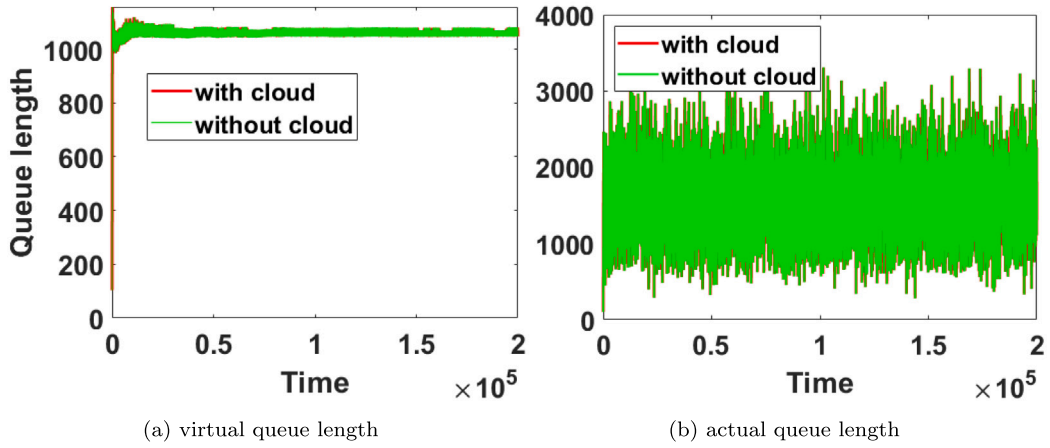


Fig. 2. Aggregate queue length evolution at the users. We have plotted the queue lengths with and without the cloud. However, the two graphs coincide as the queue lengths of the users remain the same with or without the cloud.

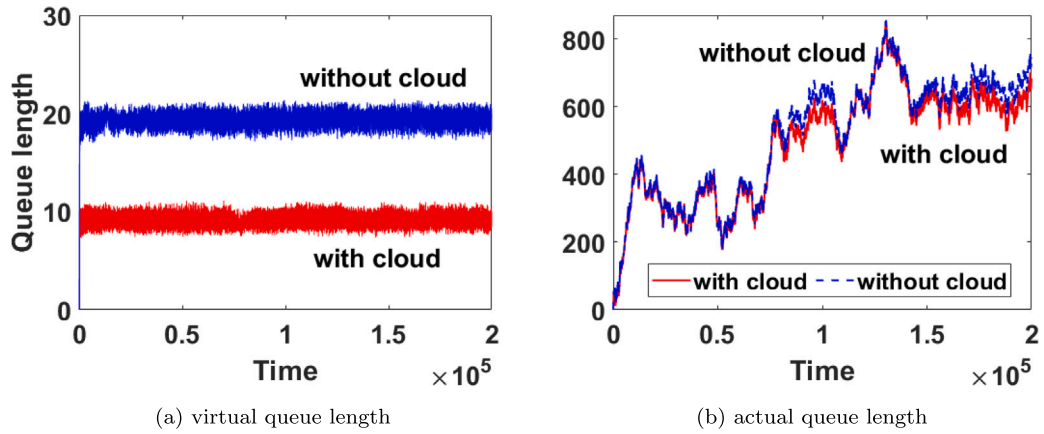


Fig. 3. Aggregate queue length evolution at the MEC server.

6.1. Throughput optimality of the MEC system without cloud

This subsection demonstrates that Algorithm 1 is throughput-optimal for the MEC system excluding the cloud (i.e., $W=\infty$). For this, we consider the arrival rate per user to be 4.9896 Packets/s and the arrival rate vector for services to be $[0.396, 0.396, 0.198, 0.7920, 0.1980]$ services/s. The traffic load constitutes 99 percent of the capacity region. We execute the simulations for 2×10^5 time slots. Figs. 2 and 3 show that the queues are stable. Since the queue length at the user is in bits and at the MEC server in service, we observe the aggregate queue length at the users in each slot is higher than at the MEC server.

6.2. Throughput optimality of the MEC system with cloud

In this subsection, we demonstrate that Algorithm 1 is throughput-optimal for the MEC system with the cloud. For this, we consider the arrival rate per user to be 8.8308 Packets/s, the arrival rate vector for services to be $[0.594, 0.792, 0.396, 0.99, 0.396]$ per s, $W = 100$. The traffic load constitutes 99 percent of the capacity region. We execute the simulations for 10^5 time slots. Fig. 4 shows that the queues are stable.

6.2.1. Comparison

We consider the same arrival rate as Section 6.1 and compare the queue length evolution of the MEC system after adding cloud (i.e., $W = 1$). Figs. 2(a) and 2(b) show that adding the cloud to the MEC system does not affect the user queue length since it cannot change the departure rate from the users' queues. However, it affects the departure to the MEC server. It is clear from Figs. 3(a) and 3(b) that the queue lengths of actual and virtual queues reduce when we add the cloud to the MEC system. The average actual queue length (i.e., averaged over 2×10^5 iterations) at the MEC server with and without the cloud are 489.07 and 511.41 services,

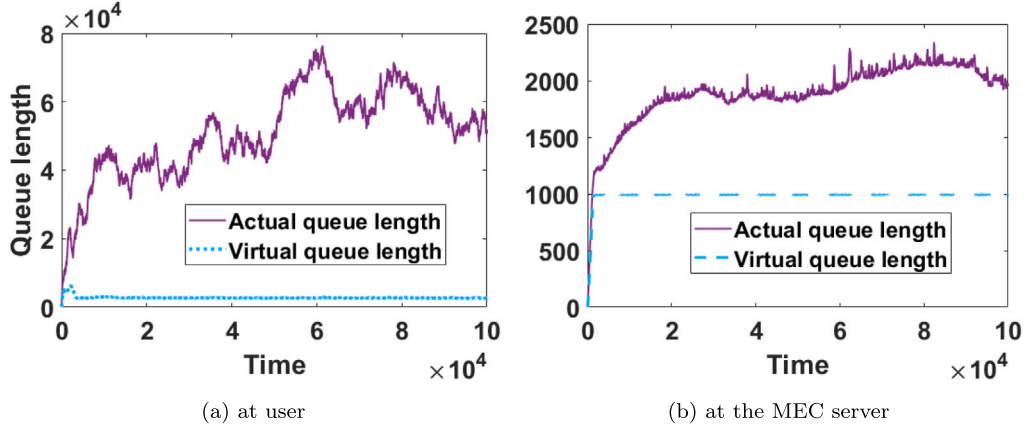
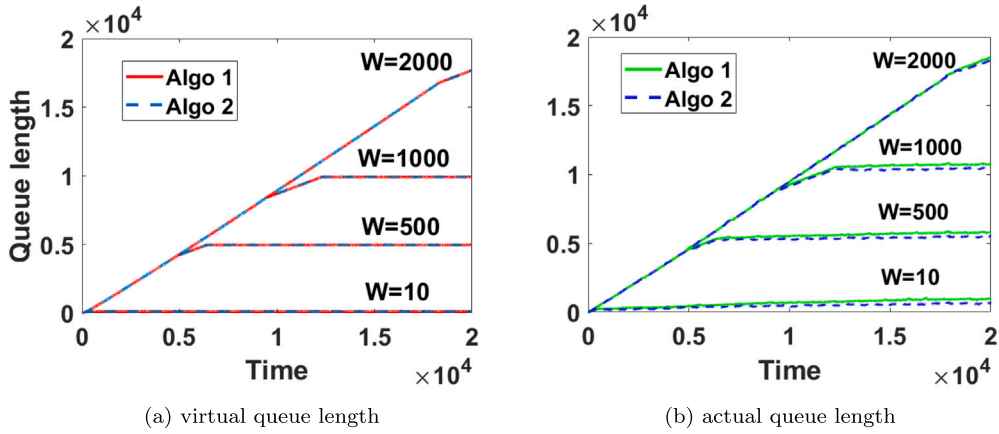


Fig. 4. Aggregate queue length evolution of the MEC system.

Fig. 5. Aggregate queue length evolution at the MEC server for different values of W .

respectively. The reduction of queue length is unsurprising since adding the cloud increases the overall capacity region if the MEC server acts as a bottleneck of the capacity region.

6.2.2. Effect of parameter W and comparison between Algorithms 1 and 2

We consider the same arrival rate as Section 6.2 and vary W from 0 to 2000, and run the simulations for 20000 time slots. We only observe the change in average cost of offloading to the cloud and the queue lengths at the MEC server since changing W does not affect the queue lengths of the users. As we mention in Section 3, observe that under Algorithm 1, increasing the parameter W increases the queue lengths in Fig. 5 and decreases the average cost of offloading to the cloud in Fig. 6. Therefore, W acts as a trade-off parameter between the average queue length and the average cost of offloading to the cloud. The same phenomenon occurs under Algorithm 2 for the same reason. Fig. 5(b) shows that the actual queue lengths are slightly higher for Algorithm 1 than for Algorithm 2. The virtual queue evolution under Algorithm 1 and Algorithm 2 are very close. But the average virtual queue length (i.e., averaged over 20000 time slots) is smaller under Algorithm 1. For example, they are 4202.1 and 4204.9 services under Algorithm 1 and 2, respectively for $W=500$. In contrast, the average costs of offloading to the cloud for Algorithms 1 and 2 are the same. Therefore, the overall performance of Algorithm 2 is better than Algorithm 1.

6.2.3. Effect of ϵ

This subsection demonstrates how the parameter ϵ affects the queue lengths. The virtual queues in Figs. 7(a) and 8(a) are stable for $\epsilon=0.01, 0.0001$ and grow linearly with time for $\epsilon=0.5$. The reason is that $(1+\epsilon)\lambda \in C$ for the values of $\epsilon=0.01, 0.0001$ and $(1+\epsilon)\lambda \notin C$ for $\epsilon=0.5$. The actual queues are stable when the virtual queues are stable (i.e., for $\epsilon=0.01, 0.0001$). Since larger ϵ implies more arrivals to the virtual queues where the capacity region is the same, the average queue length of the virtual queues for $\epsilon=0.01$ is larger than for $\epsilon=0.0001$. However, the average queue length of the actual queues for $\epsilon=0.01$ is smaller than for $\epsilon=0.0001$ (see Figs. 7(b) and 8(b)). The stability of the virtual queues ensures that the service rate of the virtual queues is at least $(1+\epsilon)\lambda$. Since the service rate is the same for both the virtual and actual queues, the guaranteed service rate increases as we increase ϵ for a given arrival rate. Figs. 7(b) and 8(b) show that the actual queues can be stable or unstable when the virtual queues are unstable, e.g., for $\epsilon = 0.5$.

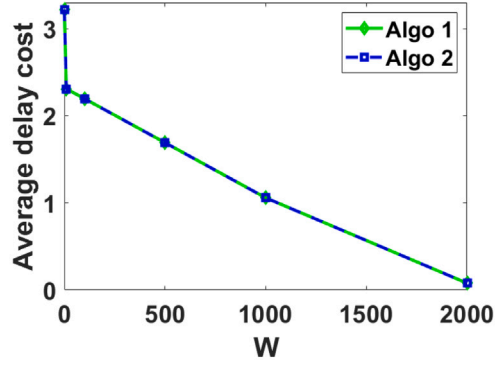


Fig. 6. Average cost of offloading to the cloud for different values of W .

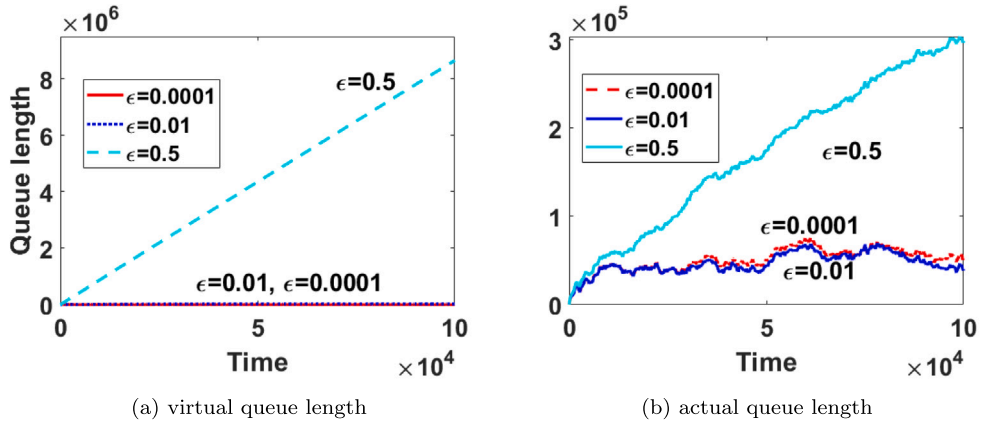


Fig. 7. Aggregate virtual queue length evolution at the users. The graphs for virtual queue lengths for $\epsilon = 0.01$ and $\epsilon = 0.0001$ coincide due to the considerably larger queue length for $\epsilon = 0.5$. Additionally, the virtual queue length for $\epsilon = 0.01$ is marginally higher than for $\epsilon = 0.0001$.

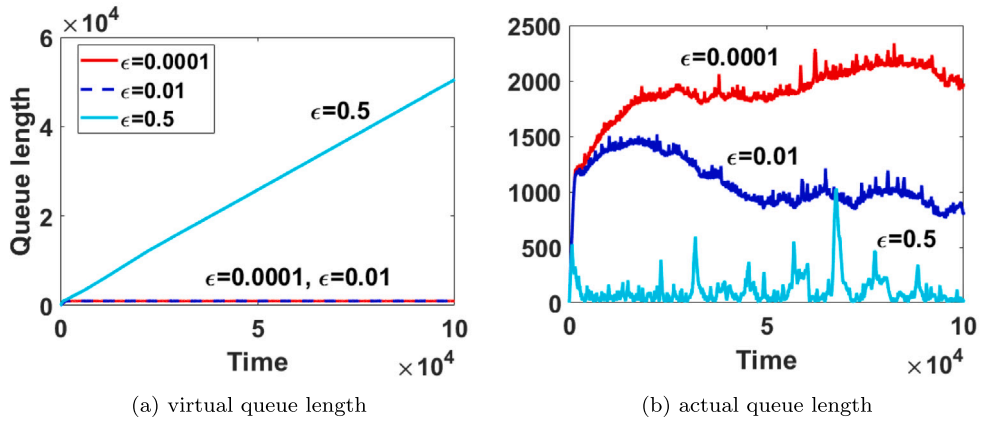


Fig. 8. Aggregate actual queue length evolution at the MEC server. The graphs for virtual queue lengths for $\epsilon = 0.01$ and $\epsilon = 0.0001$ coincide due to the considerably larger queue length for $\epsilon = 0.5$. Additionally, the virtual queue length for $\epsilon = 0.01$ is marginally higher than for $\epsilon = 0.0001$.

6.3. Comparison with BP-based drift-plus-penalty [12,13]

We consider the same arrival rate as in Section 6.2 and plot the average queue lengths vs. costs for different W . We run the simulations for 20000 time slots. The average queue length is calculated, averaging the users' and MEC server's total queue

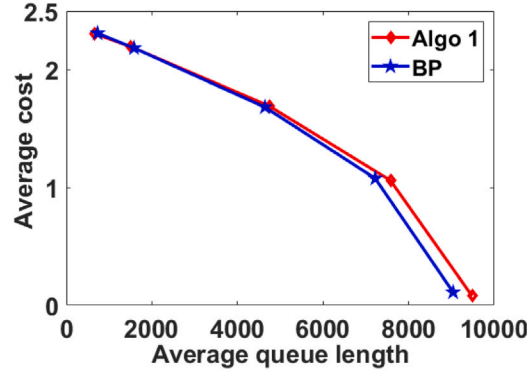


Fig. 9. Average cost of offloading to the cloud vs. queue length for different values of W .

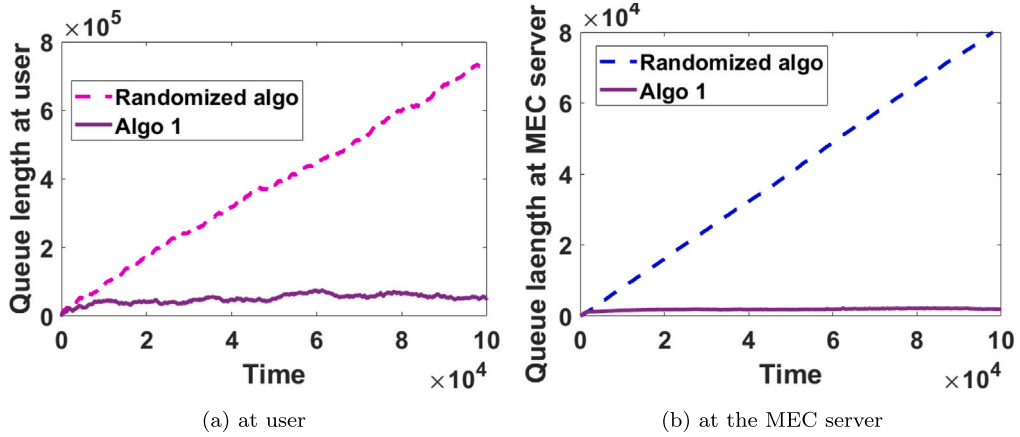


Fig. 10. Aggregate actual queue length evolution of the MEC system.

lengths over 20000 time slots. We vary W from 10 to 2000 and from 4×10^5 to 1.2×10^7 for Algorithm 1 and BP based drift-plus-penalty algorithm, respectively. We note that Algorithm 1 offers almost same performance as BP-based drift-plus-penalty (see Fig. 9). However, we need to employ different W in the two algorithms for similar queue length-cost pairs. As we demonstrated in Section 6.2, Algorithms 1 and 2 offer very close performance, so we need not compare Algorithm 2 and BP-based drift-plus-penalty.

6.4. Comparison with randomized algorithm

We compare the queue lengths of Algorithm 1 and randomized algorithm in Figs. 10(a) and 10(b). For this, we consider the same parameters as Section 6.2 and run the simulations for 100000 slots. The randomized algorithm schedules a user from the set of users randomly for packet transmission and selects a configuration for the MEC server randomly from the set $\mathcal{N}$ (see Section 2.4) for service execution in each slot. The actual queue lengths at the users and MEC server for Algorithm 1 are much lower than for the randomized algorithm, and it does not stabilize the queues both at users (Fig. 10(a)) and MEC server (Fig. 10(b)). Moreover, the cost of offloading to the cloud for Algorithm 1 (2.37) is less than for the randomized algorithm (2.52). This difference implies that Algorithm 1 achieves more efficient resource allocation and average cost optimization in offloading tasks to the cloud.

6.5. Comparison of performance of Algorithms 1 and 3

In this subsection, we compare the queue length evolution between Algorithms 1 and 3. For this, we consider an MEC system without cloud and the same parameters as Section 6.1. The aggregate virtual queue length of Algorithm 3 is more than that of

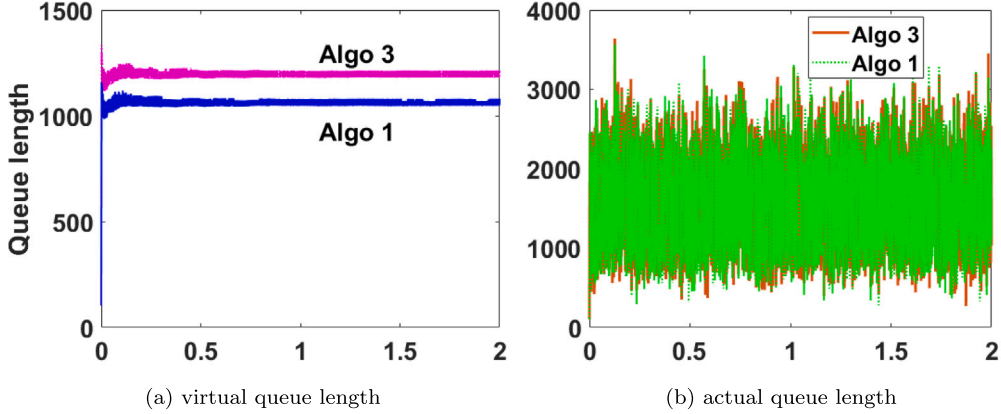


Fig. 11. Aggregate queue length evolution at Users. We have plotted the queue lengths for Algorithms 1 and 3. However, the two graphs coincide in Fig. 11(b) as the actual queue lengths of the users are almost the same for Algorithms 1 and 3.

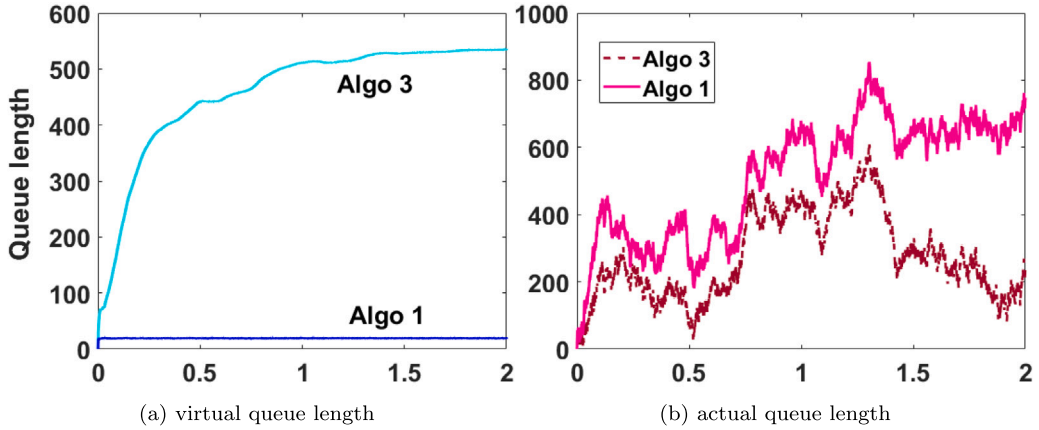


Fig. 12. Aggregate queue length evolution at the MEC server.

Algorithm 1 (Figs. 11(a) and 12(a)) because arrivals to the virtual queues are more under Algorithm 3. However, the aggregate actual queue length of the MEC server reduces significantly while the aggregate actual queue length of the users remains almost same (Figs. 11(b) and 12(b)). Altogether, the aggregate actual queue lengths improve under Algorithm 3.

6.6. A MEC system with local computation

In this subsection, we consider an MEC system with local computation and evaluate the performance of Algorithm 4. We consider the same parameters as Section 6 except for the CPU frequency of the MEC server and the uplink rate of the users. The former is set to 10 GHz [34], and the latter is 365 and 298 Mbps for two different channel states. We consider that each user has a CPU of frequency 4 GHz [13]. We consider the arrival rate per user to be 95.3964 Packets/s, the arrival rate vector for services to be [8.91, 5.74, 4.554, 8.5140, 4.7520] per s, $W_1 = 1$, and $W = 10$. We note that the traffic load constitutes 99 percent of the capacity region.

6.6.1. Throughput optimality of the MEC system

We plot the aggregate queue lengths of users' MAC layer queues, application layer queues at the users, and MEC server's queues. For this, we execute the simulations for 10^5 time slots. Figs. 13(a), 13(b), and 13(c) show that Algorithm 4 stabilizes users' application layer queues, MAC layer queues, and MEC server's queues are stable, respectively.

6.6.2. Effect of W

In this subsection, we vary W from 10 to 35 keeping W_1 fixed to 1, and run the simulations for 10^5 time slots. We observe the change in average offloading costs and average queue lengths for different values of W . As we increase W , the number of packets sent to the cloud decreases (see the offloading rule at the MEC server in Algorithm 4). Consequently, this increases the MEC server's

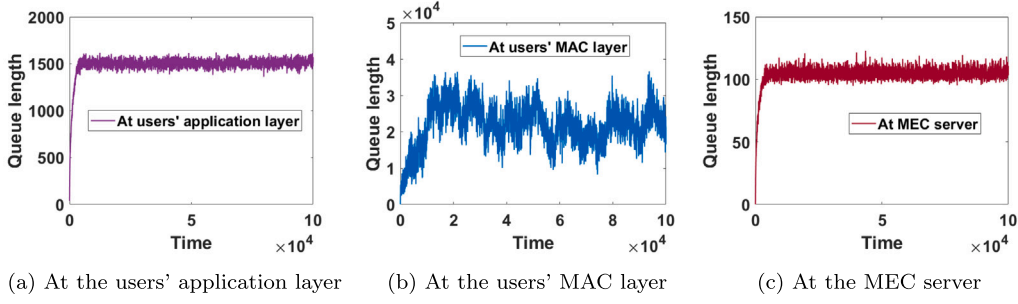
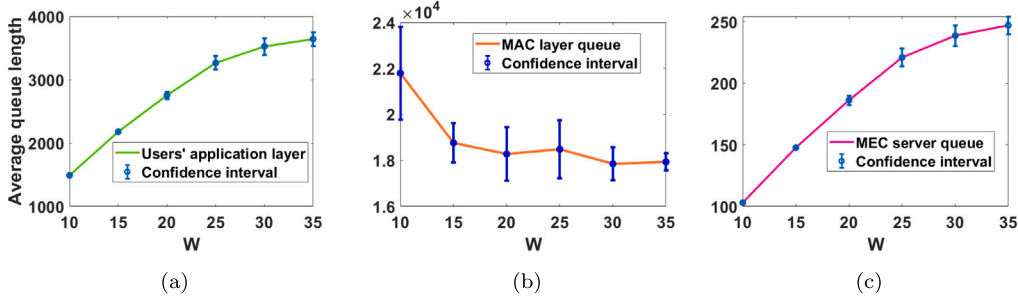
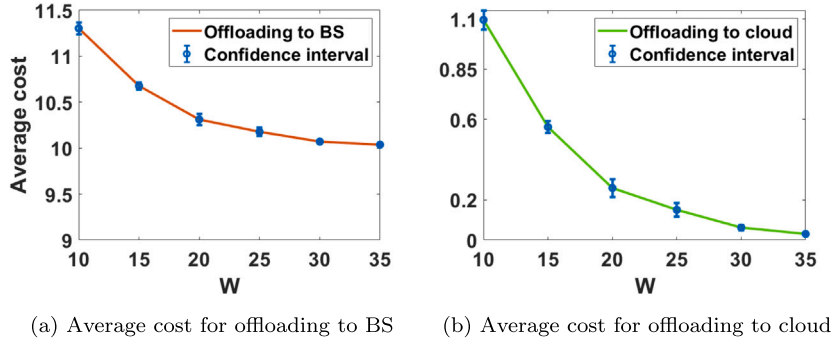


Fig. 13. Aggregate queue length evolution of the MEC system with local computation.

Fig. 14. Average queue length at different layers for different values of W .Fig. 15. Average cost for different values of W .

average queue length (see Fig. 14(c)) and reduces the average cost of offloading to the cloud (see Fig. 15(b)). Since the MEC server's queues create back-pressure at the user's application layer (see the offloading rule at the users' application layer in Algorithm 4), increasing the MEC server's queue length decreases the number of services sent to the users' MAC layer from the application layer. Consequently, this increases the users' average queue length at the application layer (see Fig. 14(a)), reduces the average offloading cost to the BS (see Fig. 15(a)), and reduces the average number of arrivals to the MAC layer queues. Although the average number of arrivals to the MAC layer queues decreases, the queue length at the MAC layer might follow a different pattern due to the weighting of queues by the data size of services. We observe that the users' average queue length at the MAC layer decreases initially from $W = 10$ to 20 and then increases slightly at $W = 25$ (see Fig. 14(b)) and decreases further. We note that the average queue length at the MAC layer is measured in packets. For parameters $W = 20$ and $W = 25$, the average number of services in the MAC layer is 277 and 259, respectively. This indicates the number of services in the MAC decreases as we increase W . However, the MAC layer queues are weighted by the data size of services. Hence, the average queue length at the MAC layer might not follow a decreasing pattern as the average number of services at the MAC layer. The plots also include 95% confidence intervals based on 10 different datasets.

6.6.3. Effect of W_1

In this subsection, we vary W_1 from 1 to 35 keeping W fixed to 10, and run the simulations for 10^5 time slots. We observe the change in average offloading costs and average queue lengths for different values of W_1 . When we increase the value of W_1 from

Table 3
Effect of W_1 variation on average departure and queue lengths.

Values of W_1	1	10	15	20	25	35
Average departure to the MAC (# of services)	5.02	3.94	3.92	3.91	3.90	3.89
Average queue at the MAC (# of services)	602.82	167.41	180.95	190	217.65	233.56

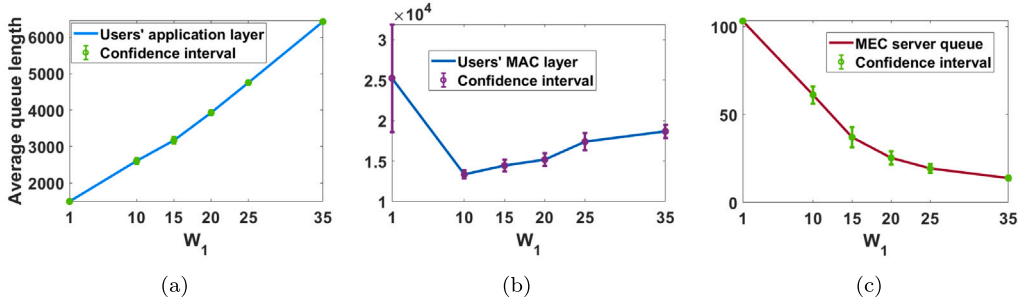


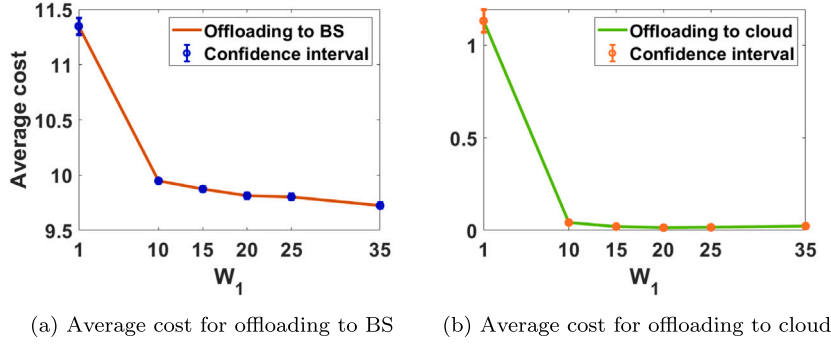
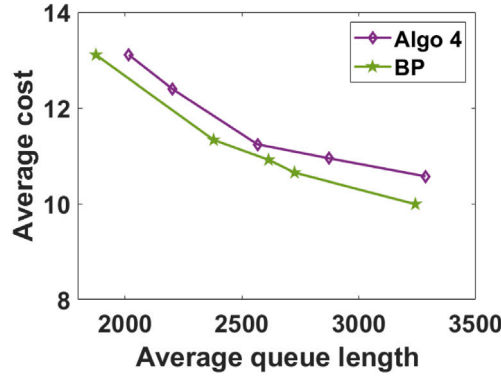
Fig. 16. Average queue length at different layers for different values of W_1 .

1 to 10, there is a sudden reduction in average number of departures to the MAC, and then the reduction in average number of departures to the MAC becomes very small as we increase W_1 further (see Table 3). Increasing W_1 reduces the number of services sent to the BS, reducing the average number of services offloaded to the cloud. The former reduces the average queue length of the MEC server. Reduction in MEC server queue length reduces the back pressure at the user's application layer, i.e., it counters the effect of increasing W_1 .

Hence, increasing W_1 will reduce the departures to the MAC layer queues as long as the effect of increasing W_1 dominates the impact of decreasing queue lengths at the MEC server. Average offloading cost follows the same trend as the average numbers of departures to the MAC layer (see Fig. 17(a)), i.e., it decreases as we increase W_1 , and so do the MEC server queues (see Fig. 16(c)) and average offloading cost to the cloud (see Fig. 17(b)). Since the average number of arrivals to the application layer of the users are the same, and the departure to the MAC layer reduces, it increases the average queue length of users' application layer queues (see Fig. 16(a)). The average cost of offloading to the cloud is very low because, when the value of W_1 exceeds 10, the average per service queue length of the MEC server falls below 20, leading to tiny numbers of departures to the cloud as for most of the services (see Table 2) the queue length has to cross the threshold (W_4), that is more than 20. As we increase W_1 , the MAC layer's average queue length behavior can be characterized by two distinct phases (see Fig. 16(b)). The average number of services offloaded from the application layer to the MAC layer decreases with an increase in W_1 . Initially, the drop is steep, because of which the average queue length at the MAC layer decreases. As we continue to increase W_1 , the offload process becomes more bursty due to increase in queue length at the users' application layer (see the offloading rule at the users' application layer in Algorithm 4), though the average offloading rate decreases marginally. Consequently, the mean arrival rate to the MAC layer decreases marginally, but the average queue length at the MAC layer starts to increase. The plots also include 95% confidence intervals based on 10 different datasets.

6.7. Comparison with BP-based algorithms [12,13]

We consider the same arrival rate as in Section 6.6 and plot the average queue lengths vs. costs for different W . We run the simulations for 10^5 time slots. We compute the average queue length by averaging the sum of users' and MEC server's total queue lengths over 100000 time slots. We vary W from 8 to 20 keeping $W_1 = 1$ and from 5 to 15 keeping $W_1 = 0.5$ for Algorithm 4 and BP-based drift-plus-penalty algorithm, respectively. The BP-based drift-plus-penalty algorithm performs slightly better than Algorithm 4 (see Fig. 18). However, as we mentioned earlier, the BP-based drift-plus-penalty algorithms are not viable in the MEC system as these require cross-layer message exchange and per-service queue length information at the MAC layer. However, we need to employ different W and W_1 in the two algorithms for similar queue length-cost pairs.

Fig. 17. Average cost for different values of W_1 .Fig. 18. Average cost of offloading vs. queue length for different values of W and W_1 .

6.8. Comparison with randomized algorithm

We compare the queue lengths of Algorithm 4 and randomized algorithm in Figs. 19(a) and 19(b). For this, we consider the same parameters as Section 6.6 and run the simulations for 10^5 time slots. The randomized algorithm randomly selects a configuration from the set $\mathcal{N}_u$ (see Section 5.1) for local computation for each user. It selects a user and a service randomly for wireless packet scheduling and a configuration from the set $\mathcal{N}$ (see Section 2.4) at the MEC server for service execution in each slot. The randomized algorithm does not stabilize the queues both at the users and at the MEC server (Figs. 19(a) and 19(b)). Moreover, the average cost for Algorithm 1 (12.4) is less than for the randomized algorithm (20). This difference implies that Algorithm 4 achieves more efficient resource allocation and average cost optimization.

7. Conclusions

We have proposed throughput optimal algorithms for task offloading and scheduling which can achieve average optimal delay arbitrarily closely without using cross layer communication. Theorems 2 and 4 establish that the proposed algorithms are throughput-optimal. Theorems 3 and 4 establish that the algorithms can yield average cost of offloading to the clouds arbitrarily close to the optimal. It is of interest to design a sub-optimal solution to the MEC configuration problem that can provide throughput-optimality or can achieve a substantial fraction of the capacity region. It also remains to investigate problems where the services require processing times of more than one slot and experience dynamic cost of offloading to the cloud when executed at the cloud. To the best of our knowledge, achieving tight bounds for the queue lengths under our framework is an open problem. It would be interesting to see if one can achieve tight bounds on the expected virtual queue lengths via considering their variance. However, achieving tight bounds for virtual queue lengths alone may not provide significant insight into the actual behavior of the MEC system because the tight bounds of actual queues characterize the system's performance. Providing tight bounds for the actual queues would be a valuable extension to our work.

CRediT authorship contribution statement

Ankita Koley: Writing – review & editing, Writing – original draft, Formal analysis, Conceptualization. **Chandramani Singh:** Supervision.

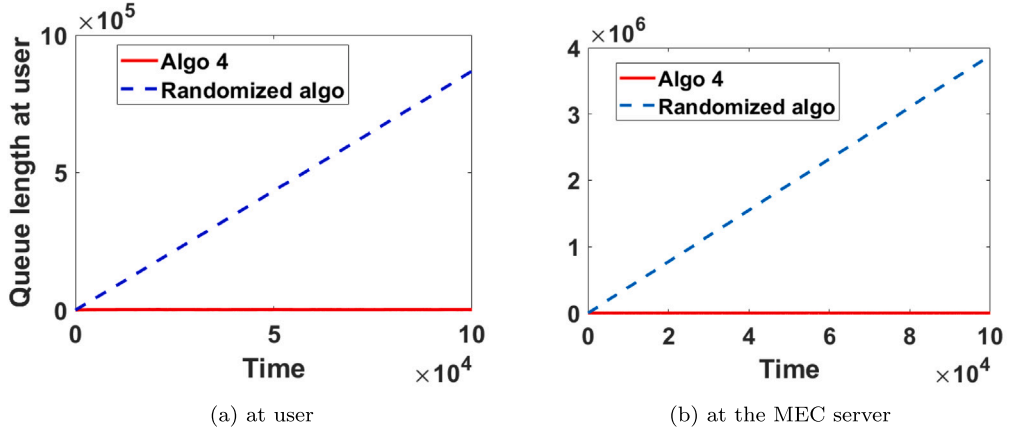


Fig. 19. Aggregate queue length evolution of the MEC system.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported jointly by Centre for Network Intelligence (a Cisco CSR initiative), Indian Institute of Science (IISc) and Aircel TCoE project 39010C.

Appendix A. Some additional notations

Let us begin by introducing some more notation. Let us first define

$$M_{u,s}(t) \triangleq \sum_{\tau=1}^t A_{u,s}(\tau). \quad (\text{A.1})$$

Let $G_j(t)$ be the cumulative number of slots until time t when the channel state is j . Let $\hat{G}_{j,u}(t)$ be the cumulative number of slots until time t when the channel state is j and user u is scheduled.

$$\sum_{u \in \mathcal{U}} \hat{G}_{j,u}(t) = G_j(t) \quad \forall j \in J. \quad (\text{A.2})$$

Let $L_u(t)$ and $\hat{L}_u(t)$ be the total departure until time t from Q_u and $\hat{Q}_u$ respectively. Let $\Psi_u(t)$ be the total scheduled departure until time t , same for both Q_u and $\hat{Q}_u$.

$$L_u(t) \leq \Psi_u(t) = \sum_{j \in J} \hat{G}_{j,u}(t) \eta_{j,u}. \quad (\text{A.3})$$

Let $\bar{L}_{u,s}(t)$ be the total departure (in terms of packets) until time t for service type s and user u . Let $F_{u,s}(t)$ be the cumulative arrival to BS until time t for service type s and user u . Subsequently, we obtain the relationships between $F_{u,s}(t)$ and $\bar{L}_{u,s}(t)$.

$$F_{u,s}(t) = \sum_{\tau=1}^t V_{u,s}(\tau) = \left\lfloor \frac{\bar{L}_{u,s}(t)}{d_s} \right\rfloor, \quad (\text{A.4})$$

$$\frac{\bar{L}_{u,s}(t)}{d_s} - 1 \leq F_{u,s}(t) \leq \frac{\bar{L}_{u,s}(t)}{d_s}. \quad (\text{A.5})$$

Let $\Psi_{BS,s}(t)$ be the cumulative scheduled departure until time t for both the queues $Q_{BS,s}$ and $\hat{Q}_{BS,s}(t)$. Then,

$$\Psi_{BS,s}(t) = \sum_{\tau=1}^t \left(N_s(\tau) + \frac{E_s^c(\tau)}{d_s} \right). \quad (\text{A.6})$$

Let $L_{BS,s}(t)$ and $\hat{L}_{BS,s}(t)$ be the total departure until time t from queue $Q_{BS,s}(t)$ and $\hat{Q}_{BS,s}$. Hence, $L_{BS,s}(t) \leq \Psi_{BS,s}(t)$ and $\hat{L}_{BS,s}(t) \leq \Psi_{BS,s}(t)$. Let $H_s^c(t)$ be the total number of packets scheduled to be sent to the cloud until time t . Hence,

$$H_s^c(t) \triangleq \sum_{\tau=1}^t E_s^c(\tau). \quad (\text{A.7})$$

$$\frac{d}{dt} \hat{p}_u(t) = \hat{a}_u(t) = (1 + \epsilon) \sum_{s \in S} \lambda_{u,s} d_s, \quad (\text{B.4})$$

$$g_j(t) = \mu_j t, \quad (\text{B.5})$$

$$g_j(t) = \sum_{u \in U} \hat{g}_{j,u}(t), \quad (\text{B.6})$$

$$\psi_u(t) = \sum_{j \in J} \hat{g}_{j,u}(t) \eta_{j,u}, \quad (\text{B.7})$$

for any interval $[t_1, t_2] \subset [0, \infty)$,

$$\hat{l}_u(t_2) - \hat{l}_u(t_1) \leq \sum_{j \in J} \eta_{j,u} (\hat{g}_{j,u}(t_2) - \hat{g}_{j,u}(t_1)), \quad (\text{B.8})$$

if $q_u(t) > 0$, for $t \in [t_1, t_2]$, then,

$$\hat{l}_u(t_2) - \hat{l}_u(t_1) = \sum_{j \in J} \eta_{j,u} (\hat{g}_{j,u}(t_2) - \hat{g}_{j,u}(t_1)), \quad (\text{B.9})$$

$$q_u(t) = q_u(0) + p_u(t) - l_u(t), \quad (\text{B.10})$$

$$\hat{q}_u(t) = \hat{q}_u(0) + \hat{p}_u(t) - \hat{l}_u(t), \quad (\text{B.11})$$

$$\frac{d}{dt} q_u(t) = a_u(t) - d_u(t), \quad (\text{B.12})$$

$$\frac{d}{dt} \hat{q}_u(t) = \hat{a}_u(t) - \hat{d}_u(t), \quad (\text{B.13})$$

$$\sum_{s \in S} \bar{l}_{u,s}(t) = l_u(t), \quad (\text{B.14})$$

$$\frac{d}{dt} q_u(t) = \begin{cases} (a_u(t) - \sum_{j \in J} \hat{g}'_{j,u}(t) \eta_{j,u}) & \text{if } q_u(t) > 0 \\ (a_u(t) - \sum_{j \in J} \hat{g}'_{j,u}(t) \eta_{j,u})^+, & \text{otherwise,} \end{cases} \quad (\text{B.15})$$

$$\frac{d}{dt} \hat{q}_u(t) = \begin{cases} (\hat{a}_u(t) - \sum_{j \in J} \hat{g}'_{j,u}(t) \eta_{j,u}) & \text{if } \hat{q}_u(t) > 0 \\ (\hat{a}_u(t) - \sum_{j \in J} \hat{g}'_{j,u}(t) \eta_{j,u})^+, & \text{otherwise,} \end{cases} \quad (\text{B.16})$$

$$f_{u,s}(t) = \frac{l_{u,s}(t)}{d_s}, \quad (\text{B.17})$$

$$p_{BS,s}(t) = \sum_{u \in U} f_{u,s}(t), \quad (\text{B.18})$$

$$q_{BS,s}(t) = q_{BS,s}(0) + p_{BS,s}(t) - l_{BS,s}(t), \quad (\text{B.19})$$

$$\hat{q}_{BS,s}(t) = \hat{q}_{BS,s}(0) + \hat{p}_{BS,s}(t) - \hat{l}_{BS,s}(t), \quad (\text{B.20})$$

$$\frac{d}{dt} q_{BS,s}(t) = a_{BS,s}(t) - d_{BS,s}(t), \quad (\text{B.21})$$

$$\frac{d}{dt} \hat{q}_{BS,s}(t) = \hat{a}_{BS,s}(t) - \hat{d}_{BS,s}(t), \quad (\text{B.22})$$

$$\frac{d}{dt} q_{BS,s}(t) = \begin{cases} (a_{BS,s}(t) - n_s(t) - \frac{e_s^c(t)}{d_s}) & \text{if } q_{BS,s}(t) > 0 \\ (a_{BS,s}(t) - n_s(t) - \frac{e_s^c(t)}{d_s})^+, & \text{otherwise,} \end{cases} \quad (\text{B.23})$$

$$\frac{d}{dt} \hat{q}_{BS,s}(t) = \begin{cases} (\hat{a}_{BS,s}(t) - n_s(t) - \frac{e_s^c(t)}{d_s}) & \text{if } \hat{q}_{BS,s}(t) > 0 \\ (\hat{a}_{BS,s}(t) - n_s(t) - \frac{e_s^c(t)}{d_s})^+, & \text{otherwise,} \end{cases} \quad (\text{B.24})$$

$$e_s^c(t) = \frac{d}{dt} h_s^c(t).$$

Proof. See Lemma 8 of [16]. $\square$

Appendix C. Proof of Theorem 2

Proof of Theorem 2 is organized as follows:

(a) We show that with probability 1,

$$\lim_{k \rightarrow \infty} \frac{1}{z_{n_k}} \left[\frac{1}{z_{n_k} t + 1} \|P^{(z_{n_k})}(z_{n_k} t)\|_1 \right] = 0.$$

(b) For any $\zeta > 0$ there exists a finite time T such that

$$\sum_u \hat{q}_u(t) < \zeta \text{ for any time } t \geq T.$$

(c) For any $\zeta_1 > 0$ there exists a finite time T_1 such that

$$\sum_s \hat{q}_{BS,s}(t) < \zeta_1 \text{ for any time } t \geq T_1.$$

(d) using the above results for virtual queues we show that there exists a finite time T_2 such that

$$\sum_u q_u(t) + \sum_s q_{BS,s}(t) = 0 \text{ for any time } t \geq T_2.$$

To prove part (a), we use the following relation from Section 2.1, $P_u(t) = \sum_{\tau=1}^t \sum_{s \in S} A_{u,s}(\tau) d_s$. Hence, we can write from (A.1),

$$\frac{1}{z_{n_k}} \sum_{u \in \mathcal{U}'} P_u^{(z_{n_k})}(z_{n_k} t) = \frac{1}{z_{n_k}} \sum_{u \in \mathcal{U}'} \sum_{s \in S} M_{u,s}^{(z_{n_k})}(z_{n_k} t) d_s. \quad (\text{C.1})$$

Since the total arrival to the MEC server's actual queues until time t is lesser compared to the sum of total arrival to the users' actual queues and initial queue length, we can write the following:

$$\frac{1}{z_{n_k}} \sum_{s \in S} P_{BS,s}^{(z_{n_k})}(z_{n_k} t) \leq \frac{1}{z_{n_k}} \sum_{u \in \mathcal{U}'} \sum_{s \in S} (M_{u,s}^{(z_{n_k})}(z_{n_k} t) + Q_{u,s}(0)). \quad (\text{C.2})$$

Combining (C.1) and (C.2) with (B.1) we get,

$$\lim_{k \rightarrow \infty} \frac{1}{z_{n_k}} \|P^{(z_{n_k})}(z_{n_k} t)\|_1 \leq \sum_{s \in S} \sum_{u \in \mathcal{U}'} \lambda_{u,s} (1 + d_s) t + \sum_{u \in \mathcal{U}'} Q_{u,s}(0)$$

with probability 1. Hence,

$$\lim_{k \rightarrow \infty} \frac{1}{z_{n_k}} \left[\frac{1}{z_{n_k} t + 1} \|P^{(z_{n_k})}(z_{n_k} t)\|_1 \right] = 0 \text{ with probability 1.} \quad (\text{C.3})$$

Therefore, part (a) is established.

To prove part (b), we consider the Lyapunov function, $\hat{V}_1(\hat{q}(t)) = \frac{1}{2} \sum_{u \in \mathcal{U}'} \hat{q}_u^2(t)$ and derivative of the Lyapunov function, $\frac{D^+}{dt} \hat{V}_1(\hat{q}(t)) = \lim_{\delta \rightarrow 0} \frac{\hat{V}_1(\hat{q}(t+\delta)) - \hat{V}_1(\hat{q}(t))}{\delta}$.

$$\begin{aligned} \frac{D^+}{dt} \hat{V}_1(\hat{q}(t)) &= \sum_{u \in \mathcal{U}'} \hat{q}_u(t) \frac{d}{dt} \hat{q}_u(t) \stackrel{(a)}{=} \sum_{u \in \mathcal{U}'} \hat{q}_u(t) (\hat{a}_u(t) - \sum_{j \in J} \hat{g}'_{j,u}(t) \eta_{j,u}) \\ &\stackrel{(b)}{=} \sum_{u \in \mathcal{U}'} \hat{q}_u(t) \left(\sum_{s \in S} (1 + \epsilon) \lambda_{u,s} d_s - \sum_{j \in J} \eta_{j,u} \hat{g}'_{j,u}(t) \right) \end{aligned} \quad (\text{C.4})$$

where (a) follows from (B.16) and (b) follows from (B.3) and (B.4). From (B.5) we can write, $\sum_{u \in \mathcal{U}'} \hat{g}'_{j,u}(t) = g'_j(t) = \mu_j$,¹ and it further follows from [24] that $\max_u \hat{q}_u(t) \eta_{j,u} > \hat{q}_u(t) \eta_{j,u}$ for any $u \in \mathcal{U}'$ and $\hat{g}'_{j,u}(t) = 0 \forall j \in J$ and $u \in \mathcal{U}'$. Let us define $u^*(t) \triangleq \arg \max_u \hat{q}_u(t) \eta_{j,u}$.

Hence, $\hat{g}'_{j,u^*(t)}(t) = \mu_j$ and $\hat{g}'_{j,u}(t) = 0$ for $u \neq u^*(t)$. This implies that $\sum_{u \in \mathcal{U}'} \hat{q}_u(t) \sum_{j \in J} \eta_{j,u} \hat{g}'_{j,u}(t) \stackrel{(a)}{=} \sum_{j \in J} \hat{q}_{u^*(t)}(t) \eta_{j,u^*(t)} \hat{g}'_{j,u^*(t)}(t)$. We use the equality (a) to rewrite (C.4) as follows:

$$\begin{aligned} \frac{D^+}{dt} \hat{V}_1(\hat{q}(t)) &= \sum_{u \in \mathcal{U}'} \left(\hat{q}_u(t) (1 + \epsilon) \sum_{s \in S} \lambda_{u,s} d_s - \sum_{j \in J} \hat{q}_{u^*(t)}(t) \eta_{j,u^*(t)} \hat{g}'_{j,u^*(t)}(t) \right) \\ &= \sum_{u \in \mathcal{U}'} \left(\hat{q}_u(t) (1 + \epsilon) \sum_{s \in S} \lambda_{u,s} d_s - \sum_{j \in J} \hat{q}_{u^*(t)}(t) \eta_{j,u^*(t)} \mu_j \right) \\ &\leq \sum_{u \in \mathcal{U}'} \hat{q}_u(t) \left((1 + \epsilon) \sum_{s \in S} \lambda_{u,s} d_s - \sum_{j \in J} \mu_j \sum_{u \in \mathcal{U}'} \hat{q}_u(t) \eta_{j,u} \beta_{j,u} \right) \\ &= \sum_{u \in \mathcal{U}'} \left(\hat{q}_u(t) (1 + \epsilon) \sum_{s \in S} \lambda_{u,s} d_s - \sum_{j \in J} \mu_j \eta_{j,u} \beta_{j,u} \right) \text{ where, } \sum_{j \in J} \beta_{j,u} \leq 1. \end{aligned}$$

Suppose, $(1 + \epsilon) \lambda \in C$, then $\exists$ a $\beta = \{\beta_{j,u} : j \in J, u \in \mathcal{U}'\}$ such that $(1 + \epsilon) \sum_{s \in S} \lambda_{u,s} d_s < \sum_{j \in J} \mu_j \eta_{j,u} \beta_{j,u} \forall u$. For any $\zeta_1 > 0 \exists$ a $\zeta_3 > 0$ such that $\hat{V}_1(\hat{q}(t)) > \zeta_1$ implies that $\max_{u \in \mathcal{U}'} \hat{q}_u(t) \geq \zeta_3$. It further implies that the derivative of the Lyapunov function,

$$\frac{D^+}{dt} \hat{V}_1(\hat{q}(t)) \leq -\zeta_3 \min_u \left(\sum_{j \in J} \mu_j \eta_{j,u} \beta_{j,u} - (1 + \epsilon) \sum_{s \in S} \lambda_{u,s} d_s \right) = -\zeta_2 < 0.$$

Therefore, it immediately follows that for any $\zeta_1 > 0$ whenever $\hat{V}_1(\hat{q}(t)) > \zeta_1 \exists$ a ζ_2 such that derivative of the Lyapunov function, $\frac{D^+}{dt} \hat{V}_1(\hat{q}(t)) < -\zeta_2 \forall t > 0$. So, for any $\zeta > 0$ there exists a finite time T such that $\sum_u \hat{q}_u(t) < \zeta$ for any time $t \geq T$.

To prove part (c), we need the following lemmas (Lemmas 10 and 11).

¹ $x' = \frac{d}{dt} x(t)$.

Lemma 10. For all (scaled) time $t > 0$, with probability one, the following relation holds when $\hat{q}_{BS,s}(t) > 0$:

$$\hat{a}_{BS,s}(t) \leq (1 + \epsilon) \left(\sum_{u \in \mathcal{U}} \lambda_{u,s} + \frac{1}{t} \right) \quad \forall s \in \mathcal{S}. \quad (\text{C.5})$$

Proof. See Lemma 9 of [16]. $\square$

Lemma 11. Whenever, $\hat{q}_{BS,s}(t) > 0$, the following inequality follows with probability 1:

$$\sum_{s \in \mathcal{S}} \hat{q}_{BS,s}(t) \hat{d}_{BS,s}(t) \geq \sum_{N \in \mathcal{N}} \alpha_N \sum_s \hat{q}_{BS,s}(t) N_s + \sum_{E^c} \phi(E^c) \sum_s \hat{q}_{BS,s}(t) \frac{E_s^c}{d_s}.$$

Proof. From (B.22) and (B.24) we have, $\hat{d}_{BS,s}(t) = n_s(t) + \frac{e_s^c(t)}{d_s}$ if $\hat{q}_{BS,s}(t) > 0$.

From the definition of fluid limit, with probability 1:

$$\begin{aligned} \sum_s \hat{q}_{BS,s}(t) n_s(t) &= \lim_{k \rightarrow \infty} \frac{1}{z_{n_k}} \sum_s \hat{Q}_{BS,s}(z_{n_k} t) N_s(z_{n_k} t), \text{ and} \\ \sum_s \hat{q}_{BS,s}(t) \frac{e_s^c(t)}{d_s} &= \lim_{k \rightarrow \infty} \frac{1}{z_{n_k}} \sum_s \frac{\hat{Q}_{BS,s}(z_{n_k} t)}{z_{n_k}} \frac{E_s^c(z_{n_k} t)}{d_s}. \end{aligned}$$

From Algorithm 1 we can write the following,

$$\sum_s \hat{Q}_{BS,s}(t) N_s(t) = \max_{N \in \mathcal{N}} \sum_s \hat{Q}_{BS,s}(t) N_s \geq \sum_{N \in \mathcal{N}} \alpha_N \sum_s \hat{Q}_{BS,s}(t) N_s. \quad (\text{C.6})$$

$$\begin{aligned} \text{Therefore, } \sum_s \hat{q}_{BS,s}(t) n_s(t) &\geq \lim_{k \rightarrow \infty} \frac{1}{z_{n_k}} \sum_{N \in \mathcal{N}} \alpha_N \sum_s \hat{Q}_{BS,s}(z_{n_k} t) N_s \\ &= \sum_{N \in \mathcal{N}} \alpha_N \sum_s \hat{q}_{BS,s}(t) N_s. \end{aligned} \quad (\text{C.7})$$

$$\text{We define, } \max_{E_s^c \leq C_{cloud}} \sum_s \hat{q}_{BS,s}(t) \frac{E_s^c}{d_s} \triangleq \sum_s \hat{q}_{BS,s}(t) \frac{e_s^{c*}}{d_s}. \quad (\text{C.8})$$

$$\begin{aligned} \text{We note that, } \sum_s \hat{Q}_{BS,s}(z_{n_k} t) \frac{E_s^c(z_{n_k} t)}{d_s} &\geq \sum_s \hat{Q}_{BS,s}(z_{n_k} t) \frac{E_s^c(z_{n_k} t)}{d_s} - W \sum_s \frac{E_s^c(z_{n_k} t)}{d_s} \Delta_s \\ &\geq \sum_s \hat{Q}_{BS,s}(z_{n_k} t) \frac{E_s^c(z_{n_k} t)}{d_s} - W \sum_s \frac{E_s^c(z_{n_k} t)}{d_s} \Delta_s \\ &= \max_{E_s^c \leq C_{cloud}} \sum_s \hat{Q}_{BS,s}(z_{n_k} t) \frac{E_s^c}{d_s} - W \sum_s \frac{E_s^c}{d_s} \Delta_s \\ &\geq \sum_s \hat{Q}_{BS,s}(z_{n_k} t) \frac{e_s^{c*}}{d_s} - W \sum_s \frac{e_s^{c*}}{d_s} \Delta_s. \end{aligned}$$

Taking limit on the both sides after dividing by z_{n_k} , we get,

$$\begin{aligned} \lim_{k \rightarrow \infty} \sum_s \frac{\hat{Q}_{BS,s}(z_{n_k} t)}{z_{n_k}} \frac{E_s^c(z_{n_k} t)}{d_s} &\geq \lim_{k \rightarrow \infty} \sum_s \frac{\hat{Q}_{BS,s}(z_{n_k} t)}{z_{n_k}} \frac{e_s^{c*}}{d_s} - W \frac{\sum_s \frac{e_s^{c*}}{d_s} \Delta_s}{z_{n_k}} \\ &\stackrel{(a)}{=} \sum_s \hat{q}_{BS,s}(t) \frac{e_s^{c*}}{d_s} \geq \sum_{E^c} \phi(E^c) \sum_s \hat{q}_{BS,s}(t) \frac{E_s^c}{d_s}. \end{aligned} \quad (\text{C.9})$$

The equality (a) in (C.9) follows from the fact that $\lim_{k \rightarrow \infty} W \frac{\sum_s \frac{e_s^{c*}}{d_s} \Delta_s}{z_{n_k}} = 0$ with probability 1. The lemma follows after combining (C.7) and (C.9). $\square$

With Lemma 11 we proceed further to prove part (c). Consider another Lyapunov function $\hat{V}_2(\hat{q}(t)) = \frac{1}{2} \sum_s \hat{q}_{BS,s}^2(t)$ and derivative of the Lyapunov function, $\frac{D^+}{dt} \hat{V}_2(\hat{q}(t)) = \lim_{\delta \rightarrow 0} \frac{\hat{V}_2(\hat{q}(t+\delta)) - \hat{V}_2(\hat{q}(t))}{\delta}$.

$$\frac{D^+}{dt} \hat{V}_2(\hat{q}(t)) = \sum_{s \in \mathcal{U}} \hat{q}_{BS,s}(t) \frac{d}{dt} \hat{q}_{BS,s}(t) \stackrel{(a)}{=} \sum_{s \in \mathcal{U}} \hat{q}_{BS,s}(t) (\hat{a}_{BS,s}(t) - \hat{d}_{BS,s}(t))$$

$$\begin{aligned}
&\leq \sum_{s \in S} \hat{q}_{BS,s}(t)(1+\epsilon) \left(\sum_{u \in U} \lambda_{u,s} + \frac{1}{t} \right) - \sum_{N \in \mathcal{N}} \alpha_N \sum_s \hat{q}_{BS,s}(t) N_s - \sum_{E^c \in \mathcal{E}^c} \phi(E^c) \sum_s \hat{q}_{BS,s}(t) \frac{E_s^c}{d_s} \quad [\text{from Lemmas 10 and 11}] \\
&= \sum_{s \in S} \hat{q}_{BS,s}(t) \left((1+\epsilon) \left(\sum_{u \in U} \lambda_{u,s} + \frac{1}{t} \right) - \sum_{N \in \mathcal{N}} \alpha_N N_s - \sum_{E^c} \phi(E^c) \frac{E_s^c}{d_s} \right)
\end{aligned}$$

where (a) follows from (B.22). Suppose, $(1+\epsilon)\lambda \in \mathcal{C}$, then there exists a positive number β_1 and a positive integer T' such that $(1+\epsilon)(\sum_{u \in U} \lambda_{u,s} + \frac{1}{t}) < \sum_{N \in \mathcal{N}} \alpha_N N_s + \sum_{E^c} \phi(E^c) \frac{E_s^c}{d_s} \quad \forall t \geq T'$, where $\beta_1 = \sum_{N \in \mathcal{N}} \alpha_N N_s + \sum_{E^c} \phi(E^c) \frac{E_s^c}{d_s} - (1+\epsilon) \sum_{u \in U} \lambda_{u,s} > 0$ and $T' > \frac{(1+\epsilon)}{\beta_1}$. Moreover, for any $\zeta_4 > 0 \exists$ a $\zeta_2 > 0$ such that $\hat{V}_2(\hat{q}(t)) > \zeta_4$ implies $\max_s \hat{q}_{BS,s}(t) \geq \zeta_2$. Therefore we can write an upper bound for the derivative of the Lyapunov function,

$$\frac{D^+}{dt} \hat{V}_2(\hat{q}(t)) \leq -\zeta_2 \left(\sum_{N \in \mathcal{N}} \alpha_N N_s + \sum_{E^c} \phi(E^c) \frac{E_s^c}{d_s} - (1+\epsilon) \left(\sum_{u \in U} \lambda_{u,s} + \frac{1}{t} \right) \right) \leq -\zeta_3 \quad \text{for some } \zeta_3 > 0.$$

Similar to part (b) it follows that for any $\zeta > 0$ there exists a finite time T_1 such that $\sum_u \hat{q}_{BS,s}(t) < \zeta$ for any time $t \geq T_1$.

To prove part (d), we need the following Lemma:

Lemma 12. *There exists a finite time $T^* \geq \max\{T, T_1\}$ such that $\sum_u q_u(t) + \sum_s q_s(t) = 0$ for any time $t \geq T^*$.*

Proof. See Lemma 13 of [16].

Therefore, we can find a large enough $T^* > 0$ such that for any $\xi' > 0$ we

$$\begin{aligned}
&\text{have, } \lim_{k \rightarrow \infty} \left[\frac{1}{z_{n_k}} \|\mathcal{Z}^{(z_{n_k})}(z_{n_k} T)\|_1 \right] \\
&= \lim_{k \rightarrow \infty} \frac{1}{z_{n_k}} \left(\|Q^{(z_{n_k})}(z_{n_k} T)\|_1 + \|\hat{Q}^{(z_{n_k})}(z_{n_k} T)\|_1 + \frac{1}{z_{n_k} T + 1} \|P^{(z_{n_k})}(z_{n_k} T)\| \right) \\
&= \|q(T)\|_1 + \|\hat{q}(T)\|_1 \leq \xi' \triangleq 1 - \xi < 1 \text{ almost surely.}
\end{aligned}$$

Hence, by uniform integrability of the scaled sequence of $\{\frac{1}{x} \mathcal{Z}^{(x)}(x T^*) : x = 1, 2, \dots\}$ we have [16], $\limsup_{z \rightarrow \infty} \mathbb{E}[\frac{1}{z} \|\mathcal{Z}^z(z T)\|_1] \leq 1 - \xi$.

Thus the MEC system is stable under the Algorithm 1 for any arrival rate vector λ such that $(1+\epsilon)\lambda \in \mathcal{C}$.

Appendix D. Proof of Theorem 3

To prove Theorem 3, we need the following lemma.

Lemma 13.

$$\limsup_{T \rightarrow \infty} \sum_{t=1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \left(\frac{\sum_u \sum_{\tau=1}^t A_{u,s}(\tau)}{t} - \sum_u \lambda_{u,s} \right) \right] \leq 0.$$

Proof. Since the MEC system is stable, from Theorem 3.1 of [23] we can define a petite set $B = \{z \in \mathcal{Z} : \|z\|_1 \leq \kappa\}$ for some $\kappa > 0$ and $\exists$ a $T > 0$ such that $\sup_{x \in B} \mathbb{E}_x[\tau_B(T)] < \infty$, where $\tau_B(T) = \inf\{t \geq T : \mathcal{Z}(t) \in B\}$. This implies $\sum_{s \in S} \hat{Q}_{BS,s}(t) \leq \kappa + \tau_B(T)(1+\epsilon)A_{\max}$ and hence, $\mathbb{E} \left[\sum_{s \in S} \hat{Q}_{BS,s}(t) \right] \leq \kappa + \mathbb{E}[\tau_B(T)](1+\epsilon)A_{\max} < \kappa_1$ for some $\kappa_1 > 0$. Since the arrivals satisfy SLLN, fix a $\delta > 0$, $\exists$ a T_δ such that $\forall t \geq T_\delta \left| \frac{\sum_{\tau=1}^t \sum_u A_{u,s}(\tau)}{t} - \sum_u \lambda_{u,s} \right| \leq \delta$ with probability 1. That further implies, $\frac{\sum_{\tau=1}^t \sum_u A_{u,s}(\tau)}{t} \leq \delta + \sum_u \lambda_{u,s}$ w.p.1. Therefore,

$$\sum_s \hat{Q}_{BS,s}(t) \frac{\sum_{\tau=1}^t \sum_u A_{u,s}(\tau)}{t} \leq \sum_s \hat{Q}_{BS,s}(t)(\delta + \sum_u \lambda_{u,s}) \text{ w.p.1,}$$

which further implies,

$$\mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \frac{\sum_{\tau=1}^t \sum_u A_{u,s}(\tau)}{t} \right] \leq \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t)(\delta + \sum_u \lambda_{u,s}) \right].$$

Recall the definition of $\Theta(T)$ from Section 3.3,

$$\begin{aligned}
\Theta(T) &= \sum_{t=1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \left(\frac{\sum_u \sum_{\tau=1}^t A_{u,s}(\tau)}{t} - \sum_u \lambda_{u,s} \right) \right] \\
&= \sum_{t=1}^{T_\delta} \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \left(\frac{\sum_u \sum_{\tau=1}^t A_{u,s}(\tau)}{t} - \sum_u \lambda_{u,s} \right) \right] + \sum_{t=T_\delta+1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \left(\frac{\sum_u \sum_{\tau=1}^t A_{u,s}(\tau)}{t} - \sum_u \lambda_{u,s} \right) \right] \\
&\leq B_3 + \sum_{t=T_\delta+1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \left(\sum_u \lambda_{u,s} + \delta - \sum_u \lambda_{u,s} \right) \right] \quad [\text{Since, } T_\delta \text{ is finite}]
\end{aligned}$$

$$= B_3 + \sum_{t=T_\delta+1}^T \mathbb{E} \left[\hat{Q}_{BS,s}(t) \delta \right] \leq B_3 + \kappa_1 \delta (T - T_\delta).$$

After dividing both sides of above equation by T we get, $\frac{1}{T} \Theta(T) \leq \frac{B_3}{T} + \frac{\kappa_1 \delta (T - T_\delta)}{T}$. Taking the limit on the both sides, $\limsup_{T \rightarrow \infty} \frac{1}{T} \Theta(T) \leq \kappa_1 \delta$ w.p.1 $\implies \limsup_{T \rightarrow \infty} \frac{1}{T} \Theta(T) \leq 0$ as $\delta \rightarrow 0$. $\square$

To compute the bounds on average queue length and cost we define, $V(t) \triangleq \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t)$, and $\Delta V(t) \triangleq \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t+1) - \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t)$.

$$\begin{aligned} \Delta V(t) &= \frac{1}{2} \sum_s \left(\left(\hat{Q}_{BS,s}(t) - N_s(t) - \frac{E_s^c(t)}{d_s} \right)^+ + \hat{A}_{BS,s}(t) \right)^2 - \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t) \\ &\leq \frac{1}{2} \sum_s \left(\hat{A}_{BS,s}^2(t) + N_s^2(t) + \left(\frac{E_s^c(t)}{d_s} \right)^2 \right) + \sum_s \hat{Q}_{BS,s}(t) \left(\hat{A}_{BS,s}(t) - N_s(t) - \frac{E_s^c(t)}{d_s} \right) \\ &= B_2 + \sum_s \hat{Q}_{BS,s}(t) \left((1+\epsilon) \frac{\sum_{\tau=1}^t \sum_u A_{u,s}(\tau)}{t} \right) - \sum_s \hat{Q}_{BS,s}(t) N_s(t) - \sum_s \hat{Q}_{BS,s}(t) \frac{E_s^c(t)}{d_s} \end{aligned}$$

where $B_2 = SU(1+\epsilon)^2 A_{\max}^2 d_{\max}^2 + SC^2 + SC_{cloud}^2$. Taking conditional expectation conditioned on the virtual queue length vector we get,

$$\begin{aligned} \mathbb{E}[\Delta V(t) + WY(t) | \hat{\mathbf{Q}}(t)] &= \mathbb{E}[\Delta V(t) + W \Delta_s \frac{\tilde{E}_s^c(t)}{d_s} | \hat{\mathbf{Q}}(t)] \\ &\leq \mathbb{E}[\Delta V(t) + W \Delta_s \frac{E_s^c(t)}{d_s} | \hat{\mathbf{Q}}(t)] \\ &\leq B_2 + \sum_s \hat{Q}_{BS,s}(t) \mathbb{E} \left[(1+\epsilon) \frac{\sum_u \sum_{\tau=1}^t A_{u,s}(\tau)}{t} | \hat{\mathbf{Q}}(t) \right] - \sum_s (\hat{Q}_{BS,s}(t)) \mathbb{E} \left[N_s(t) + \frac{E_s^c(t)}{d_s} | \hat{\mathbf{Q}}(t) \right] + W \mathbb{E} [Y(t) | \hat{\mathbf{Q}}(t)] \\ &= B_2 + \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) (1+\epsilon) \sum_u \lambda_{u,s} | \hat{\mathbf{Q}}(t) \right] - \max_{N \in \mathcal{N}} \sum_s \hat{Q}_{BS,s}(t) N_s - \max_{E^c \in \mathcal{E}^c} \sum_s (\hat{Q}_{BS,s}(t) - W \Delta_s) \frac{E_s^c}{d_s} \\ &\quad + \sum_s \hat{Q}_{BS,s}(t) \mathbb{E} \left[(1+\epsilon) \left(\frac{\sum_u \sum_{\tau=1}^t A_{u,s}(\tau)}{t} - \sum_u \lambda_{u,s} \right) | \hat{\mathbf{Q}}(t) \right] \\ &\leq B_2 + \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) (1+\epsilon) \sum_u \lambda_{u,s} | \hat{\mathbf{Q}}(t) \right] - \sum_s \hat{Q}_{BS,s}(t) \mathbb{E} [\bar{N}_s(\bar{\pi}(t)) | \hat{\mathbf{Q}}(t)] - \left(\sum_s \hat{Q}_{BS,s}(t) \mathbb{E} \left[\frac{E_s^c(\bar{\pi}(t))}{d_s} | \hat{\mathbf{Q}}(t) \right] - W \mathbb{E} \left[\sum_s \frac{E_s^c(\bar{\pi}(t))}{d_s} \Delta_s | \hat{\mathbf{Q}}(t) \right] \right) \\ &\quad + \sum_s \hat{Q}_{BS,s}(t) \mathbb{E} \left[(1+\epsilon) \left(\frac{\sum_{\tau=1}^t \sum_u A_{u,s}(\tau)}{t} - \sum_u \lambda_{u,s} \right) | \hat{\mathbf{Q}}(t) \right]. \end{aligned} \quad (D.1)$$

From (11) and (12) in Section 3.3, we obtain $\mathbb{E} \left[\frac{E_s^c(\bar{\pi}(t))}{d_s} + N_s^{\bar{\pi}}(t) \right] \geq (1+\epsilon) \sum_u \lambda_{u,s} + \epsilon_1$ and $\sum_s \mathbb{E} \left[\frac{E_s^c(\bar{\pi}(t))}{d_s} \right] \Delta_s = Y_{opt}((1+\epsilon)\lambda + \underline{1}\epsilon_1)$. Taking expectation on the both sides of Eq. (D.1) and summing over $t = 1$ to T we obtain,

$$\begin{aligned} \mathbb{E}[V(T)] - \mathbb{E}[V(1)] + W \sum_{t=1}^T \mathbb{E}[Y(t)] \\ \leq B_2 T - \epsilon_1 \sum_{t=1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \sum_u \lambda_{u,s} \right] + W T Y_{opt}((1+\epsilon)\lambda + \underline{1}\epsilon_1) + (1+\epsilon)\Theta(T). \end{aligned} \quad (D.2)$$

Since $\mathbb{E}[V(T)] \geq 0$, we get,

$$\frac{1}{T} \sum_{t=1}^T \mathbb{E}[Y(t)] \leq \frac{B_2}{W} + Y_{opt}((1+\epsilon)\lambda + \underline{1}\epsilon_1) + \frac{(1+\epsilon)\Theta(T)}{WT} + \frac{\mathbb{E}[V(1)]}{T}.$$

After taking limits on both sides we get,

$$\begin{aligned} \limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E}[Y(t)] &\leq \frac{B_2}{W} + Y_{opt}((1+\epsilon)\lambda + \underline{1}\epsilon_1) + \limsup_{T \rightarrow \infty} \frac{(1+\epsilon)\Theta(T)}{TW} \\ &\leq \frac{B_2}{W} + Y_{opt}((1+\epsilon)\lambda + \underline{1}\epsilon_1). \end{aligned}$$

From (D.2) we can further write that $\epsilon_1 \sum_{t=1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \sum_u \lambda_{u,s} \right] \leq B_2 T + W T Y_{opt}((1+\epsilon)\lambda + \underline{1}\epsilon_1) + (1+\epsilon)\Theta(T) + \mathbb{E}[V(1)]$.

We further take limit on the both sides after dividing by T and get,

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \mathbb{E} \left[\sum_s \hat{Q}_{BS,s}(t) \sum_u \lambda_{u,s} \right] \leq \frac{B_2}{\epsilon_1} + \frac{WY_{opt}((1+\epsilon)\lambda + \frac{1}{\epsilon_1})}{\epsilon_1}.$$

Appendix E. Proof of Theorem 4

The proof of this theorem is along the same lines as the proofs of Theorems 2 and 3. However, there is a difference in the constant of the bounds provided in Theorem 3. We obtain the bound as follows:

$$\begin{aligned} & \Delta V(t) + WY(t) \\ &= \frac{1}{2} \sum_s \left(\hat{Q}_{BS,s}(t) - \hat{D}_{BS,s}(t) + \hat{A}_{BS,s}(t) \right)^2 - \frac{1}{2} \sum_s \hat{Q}_{BS,s}^2(t) + WY(t) \\ &= \frac{1}{2} \left(\sum_s 2\hat{Q}_{BS,s}(t) (\hat{A}_{BS,s}(t) - \hat{D}_{BS,s}(t)) + \left(\hat{A}_{BS,s}(t) - \hat{D}_{BS,s}(t) \right)^2 \right) + WY(t). \end{aligned} \quad (E.1)$$

We divide $\sum_s \hat{Q}_{BS,s}(t) \hat{D}_{BS,s}(t)$ into two parts as follows:

$$\begin{aligned} & \sum_s \hat{Q}_{BS,s}(t) \hat{D}_{BS,s}(t) \\ &= \sum_s \hat{Q}_{BS,s}(t) \hat{D}_{BS,s}(t) \left(\mathbb{1}_{\{\hat{Q}_{BS,s}(t) \geq C + W\Delta_{\max} + C_{cloud}\}} + \mathbb{1}_{\{\hat{Q}_{BS,s}(t) < C + W\Delta_{\max} + C_{cloud}\}} \right) \\ &= \sum_s \hat{Q}_{BS,s}(t) (N_s(t) + \frac{E_s^c(t)}{d_s}) \mathbb{1}_{\{\hat{Q}_{BS,s}(t) \geq C + W\Delta_{\max} + C_{cloud}\}} + \sum_s \hat{Q}_{BS,s}(t) \hat{D}_{BS,s}(t) \mathbb{1}_{\{\hat{Q}_{BS,s}(t) < C + W\Delta_{\max} + C_{cloud}\}} \\ &\geq \sum_s \hat{Q}_{BS,s}(t) (N_s(t) + \frac{E_s^c(t)}{d_s}) \mathbb{1}_{\{\hat{Q}_{BS,s}(t) \geq C + W\Delta_{\max} + C_{cloud}\}} + \sum_s \hat{Q}_{BS,s}(t) \left(N_s(t) + \frac{E_s^c(t)}{d_s} - C - W\Delta_{\max} - C_{cloud} \right) \mathbb{1}_{\{\hat{Q}_{BS,s}(t) < C + W\Delta_{\max} + C_{cloud}\}} \\ &\geq \sum_s \hat{Q}_{BS,s}(t) (N_s(t) + \frac{E_s^c(t)}{d_s}) - S(C + W\Delta_{\max} + C_{cloud})^2. \end{aligned} \quad (E.2)$$

Using (E.2) in (E.1) we get,

$$\begin{aligned} & \Delta V(t) + WY(t) \\ &\leq \sum_s \hat{Q}_{BS,s}(t) (\hat{A}_{BS,s}(t) - N_s(t) - \frac{E_s^c(t)}{d_s}) + \frac{1}{2} (\hat{A}_{BS,s}(t) - \hat{D}_{BS,s}(t))^2 + S(C + W\Delta_{\max} + C_{cloud})^2 + WY(t) \\ &\leq B_2 + S(C + W\Delta_{\max} + C_{cloud})^2 + \sum_s \hat{Q}_{BS,s}(t) \left(\hat{A}_{BS,s}(t) - N_s(t) - \frac{E_s^c(t)}{d_s} \right) + WY(t) \end{aligned}$$

where $B_2 = SU(1+\epsilon)^2 A_{\max}^2 d_{\max}^2 + SC^2 + SC_{cloud}^2$. The rest of the analysis follows the proof of Theorem 3. The upper bounds of the average cost and average virtual queue length are same as that of Theorem 3 except B_2 being replaced with $B_2 + S(C + W\Delta_{\max} + C_{cloud})^2$.

References

- [1] Z. Jiang, S. Mao, Energy delay tradeoff in cloud offloading for multi-core mobile devices, *IEEE Access* 3 (2015) 2306–2316, <http://dx.doi.org/10.1109/ACCESS.2015.2499300>.
- [2] Internet of Things, <http://www.audentia-gestion.fr/cisco/pdf/at-a-glance-c45-731471.pdf>, [Online].
- [3] A. Katal, S. Dahiya, T. Choudhury, Energy efficiency in cloud computing data centers: a survey on software technologies, *Cluster Comput.* (2022) 1–31.
- [4] E. Cuervo, A. Balasubramanian, D.-k. Cho, A. Wolman, S. Saroiu, R. Chandra, P. Bahl, MAUI: Making smartphones last longer with code offload, in: *Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services, MobiSys '10*, Association for Computing Machinery, New York, NY, USA, 2010, pp. 49–62, <http://dx.doi.org/10.1145/1814433.1814441>.
- [5] Q. Luo, S. Hu, C. Li, G. Li, W. Shi, Resource scheduling in edge computing: A survey, *IEEE Commun. Surv. Tutor.* 23 (4) (2021) 2131–2165.
- [6] M.J. Neely, Stochastic network optimization with application to communication and queueing systems, *Synth. Lect. Commun. Netw.* 3 (1) (2010) 1–211.
- [7] J. Xu, L. Chen, P. Zhou, Joint service caching and task offloading for mobile edge computing in dense networks, in: *IEEE INFOCOM 2018 - IEEE Conference on Computer Communications*, 2018, pp. 207–215, <http://dx.doi.org/10.1109/INFOCOM.2018.8485977>.
- [8] K. Poularakis, J. Llorca, A.M. Tulino, I. Taylor, L. Tassiulas, Service placement and request routing in MEC networks with storage, computation, and communication constraints, *IEEE/ACM Trans. Netw.* 28 (3) (2020) 1047–1060, <http://dx.doi.org/10.1109/TNET.2020.2980175>.
- [9] B. Simon, H. Mehler, A. Klein, Online learning in matching games for task offloading in multi-access edge computing, in: *ICC 2023-IEEE International Conference on Communications*, IEEE, 2023, pp. 3270–3276.
- [10] Y. Mao, J. Zhang, S.H. Song, K.B. Letaief, Power-delay tradeoff in multi-user mobile-edge computing systems, in: *2016 IEEE Global Communications Conference, GLOBECOM*, 2016, pp. 1–6, <http://dx.doi.org/10.1109/GLOCOM.2016.7842160>.
- [11] S. Bi, L. Huang, H. Wang, Y.-J.A. Zhang, Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks, *IEEE Trans. Wireless Commun.* 20 (11) (2021) 7519–7537, <http://dx.doi.org/10.1109/TWC.2021.3085319>.
- [12] Y. Cai, J. Llorca, A.M. Tulino, A.F. Molisch, Mobile edge computing network control: Tradeoff between delay and cost, in: *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*, 2020, pp. 1–6, <http://dx.doi.org/10.1109/GLOBECOM42002.2020.9322331>.
- [13] W. Fan, L. Zhao, X. Liu, Y. Su, S. Li, F. Wu, Y. Liu, Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation, *IEEE Trans. Mob. Comput.* 23 (1) (2024) 238–256, <http://dx.doi.org/10.1109/TMC.2022.3219261>.

- [14] L. Tassiulas, A. Ephremides, Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks, in: 29th IEEE Conference on Decision and Control, 1990, pp. 2130–2132 vol.4, <http://dx.doi.org/10.1109/CDC.1990.204000>.
- [15] S. Liu, E. Ekici, L. Ying, Scheduling in multihop wireless networks without back-pressure, *IEEE/ACM Trans. Netw.* 22 (5) (2014) 1477–1488, <http://dx.doi.org/10.1109/TNET.2013.2278840>.
- [16] B. Ji, C. Joo, N. Shroff, Throughput-optimal scheduling in multihop wireless networks without per-flow information, *IEEE/ACM Trans. Netw.* 21 (2) (2013) 634–647, <http://dx.doi.org/10.1109/TNET.2012.2205017>.
- [17] M. Bramson, B. D'Auria, N. Walton, Stability and instability of the maxweight policy, *Math. Oper. Res.* 46 (4) (2021) 1611–1638.
- [18] X. Ma, A. Zhou, S. Zhang, Q. Li, A.X. Liu, S. Wang, Dynamic task scheduling in cloud-assisted mobile edge computing, *IEEE Trans. Mob. Comput.* 22 (4) (2023) 2116–2130, <http://dx.doi.org/10.1109/TMC.2021.3115262>.
- [19] A. Koley, C. Singh, Optimal resource management for multi-access edge computing without using cross-layer communication, in: 2023 35th International Teletraffic Congress, ITC-35, 2023, pp. 1–9, <http://dx.doi.org/10.1109/ITC-3560063.2023.10555798>.
- [20] B. Yilmaz, A. Ortiz, A. Klein, Delay minimization for edge computing with dynamic server computing capacity: A learning approach, in: GLOBECOM 2020 - 2020 IEEE Global Communications Conference, 2020, pp. 1–6, <http://dx.doi.org/10.1109/GLOBECOM42002.2020.9322420>.
- [21] J. Kurose, K. Ross, *Computer networks: A top down approach featuring the internet*, 2017.
- [22] R. Srikant, L. Ying, *Communication Networks: an Optimization, Control, and Stochastic Networks Perspective*, Cambridge University Press, 2013.
- [23] J.G. Dai, On positive Harris recurrence of multiclass queueing networks: a unified approach via fluid limit models, *Ann. Appl. Probab.* 5 (1) (1995) 49–77.
- [24] M. Andrews, K. Kumaran, K. Ramanan, A. Stolyar, R. Vijayakumar, P. Whiting, Scheduling in a queueing system with asynchronously varying service rates, *Probab. Engrg. Inform. Sci.* 18 (2) (2004) 191–217, <http://dx.doi.org/10.1017/S0269964804182041>.
- [25] H. Kellerer, U. Pferschy, D. Pisinger, The bounded knapsack problem, *Knapsack Probl.* (2004) 185–209.
- [26] L. Tassiulas, Linear complexity algorithms for maximum throughput in radio networks and input queued switches, in: *Proceedings. IEEE INFOCOM'98, the Conference on Computer Communications. Seventeenth Annual Joint Conference of the IEEE Computer and Communications Societies. Gateway To the 21st Century* (Cat. No. 98, Vol. 2, IEEE, 1998, pp. 533–539.
- [27] S.T. Maguluri, R. Srikant, L. Ying, Stochastic models of load balancing and scheduling in cloud computing clusters, in: 2012 Proceedings IEEE Infocom, IEEE, 2012, pp. 702–710.
- [28] S.T. Maguluri, R. Srikant, L. Ying, Stochastic models of load balancing and scheduling in cloud computing clusters, in: 2012 Proceedings IEEE INFOCOM, 2012, pp. 702–710, <http://dx.doi.org/10.1109/INFCOM.2012.6195815>.
- [29] A. Kumar, D. Manjunath, J. Kuri, *Wireless Networking*, Elsevier, 2008.
- [30] H.T. Malazi, S. Clarke, Distributed service placement and workload orchestration in a multi-access edge computing environment, in: 2021 IEEE International Conference on Services Computing, SCC, 2021, pp. 241–251, <http://dx.doi.org/10.1109/SCC53864.2021.00037>.
- [31] J. Kwak, Y. Kim, J. Lee, S. Chong, DREAM: Dynamic resource and task allocation for energy minimization in mobile cloud systems, *IEEE J. Sel. Areas Commun.* 33 (12) (2015) 2510–2523, <http://dx.doi.org/10.1109/JSAC.2015.2478718>.
- [32] R. Montella, S. Kosta, D. Oro, J. Vera, C. Fernández, C. Palmieri, D. Di Luccio, G. Giunta, M. Lapegna, G. Laccetti, Accelerating linux and android applications on low-power devices through remote GPGPU offloading, *Concurr. Comput.: Pract. Exper.* 29 (24) (2017) e4286.
- [33] W.K. Seah, C.-H. Lee, Y.-D. Lin, Y.-C. Lai, Combined communication and computing resource scheduling in sliced 5G multi-access edge computing systems, *IEEE Trans. Veh. Technol.* 71 (3) (2022) 3144–3154, <http://dx.doi.org/10.1109/TVT.2021.3139026>.
- [34] W. Liu, B. Li, W. Xie, Y. Dai, Z. Fei, Energy efficient computation offloading in aerial edge networks with multi-agent cooperation, *IEEE Trans. Wireless Commun.* 22 (9) (2023) 5725–5739, <http://dx.doi.org/10.1109/TWC.2023.3235997>.